

"AI doesn't replace business architecture — it works through it."

CAPSTONE

Putting It All Together

From business requirement to a production microservices platform.

End-to-end
Spring Boot

REST and event
integration

Data and
resilience

Security and
authentication

Automated
testing

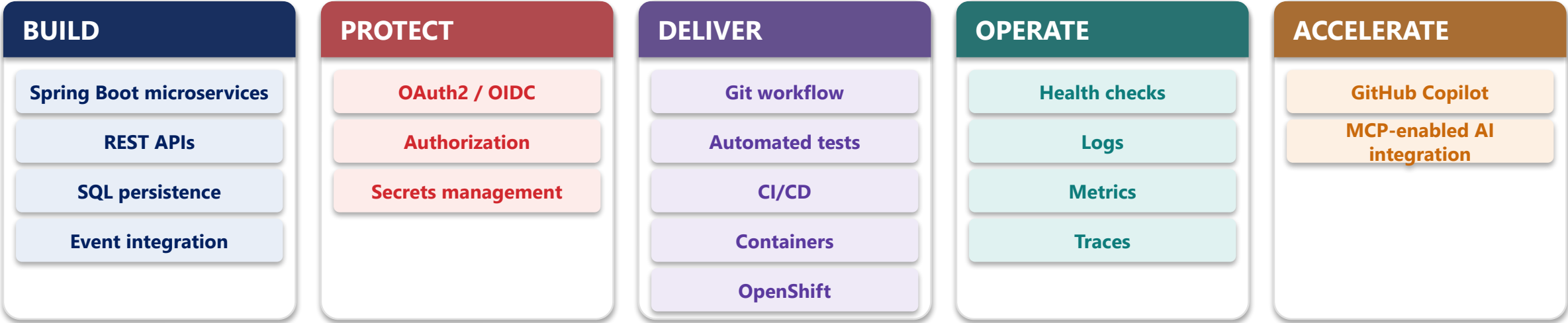
CI/CD and
containers

OpenShift and
observability

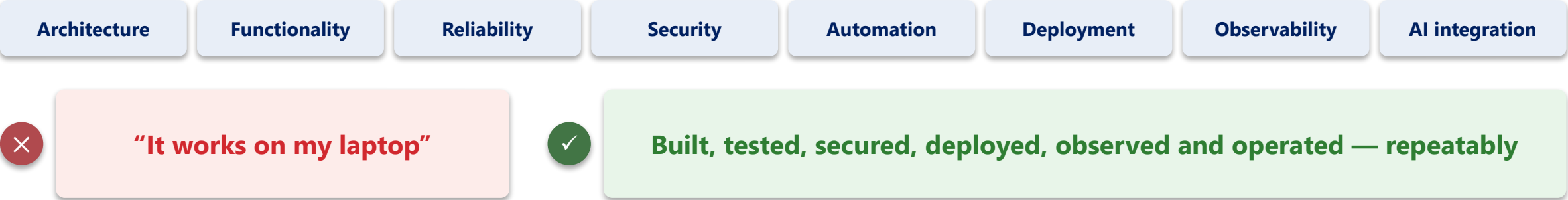
Copilot + MCP

CAPSTONE OBJECTIVES AND DELIVERABLES

What teams must build, deploy, secure and demonstrate.



EVALUATE EVERY DIMENSION



KEY TAKEAWAY Grade the whole engineering system — not just whether the code compiles.

KEY TERMS FOR THE CAPSTONE

Plain definitions for the words we'll use most.

1 **Service boundary**

The line around what a service owns — its capability, its API and its data.

2 **Synchronous integration**

A call that waits for an answer, such as a REST request.

3 **Asynchronous integration**

Publishing an event that other services process later.

4 **Transactional outbox**

Saving an event in the same transaction as the data, then publishing it.

5 **Safe fallback**

A defined outcome when a dependency fails — pending or HOLD, never an invented success.

6 **Rollback**

Returning a deployment to its previous working version.

7 **Human review**

A person decides every HOLD before any money moves.

8 **Production readiness**

Evidence that a system is secure, observable, recoverable and correct under failure.

KEY TAKEAWAY Agree on the words first — every later slide builds on them.

FULL FORMS USED IN THE CAPSTONE

Every abbreviation on these slides, spelled out.

AI	Artificial Intelligence	HTTP	HyperText Transfer Protocol	OIDC	OpenID Connect
API	Application Programming Interface	HTTPS	HyperText Transfer Protocol Secure	PR	Pull Request
CD	Continuous Delivery / Deployment	ID	Identifier	REST	Representational State Transfer
CI	Continuous Integration	JPA	Jakarta Persistence API	SHA	Secure Hash Algorithm
CI/CD	Continuous Integration / Continuous Delivery	JWT	JSON Web Token	SKU	Stock Keeping Unit
DB	Database	MCP	Model Context Protocol	SQL	Structured Query Language
DLQ	Dead-Letter Queue	OAuth	Open Authorization		
HMAC	Hash-based Message Authentication Code	OCI	Open Container Initiative		

KEY TAKEAWAY When an abbreviation isn't clear, the full form is on this slide.

FROM MODULE 12 TO THE CAPSTONE

Twelve modules, one platform — now make every piece work together.

MODULES 1–12 GAVE US

- ✓ Service boundaries and REST APIs
- ✓ Data, events and resilience
- ✓ Pipelines, images and OpenShift
- ✓ Copilot, agents and MCP
- ✓ Security, secrets and audit

NOW Design, build, deliver and demonstrate one production-ready platform.



OUR PATH THROUGH THE CAPSTONE

- 1 Scenario, architecture and contracts
- 2 Order Service and integrations
- 3 Data and resilience
- 4 Security, secrets and tests
- 5 Git, CI/CD and OpenShift
- 6 Observability and AI
- 7 The demo and readiness review

Part 1

Part 2

Part 2

Part 3

Part 4

Part 5

Part 5

KEY TAKEAWAY The Capstone isn't a new topic — it's every module working together.

PART 1: DESIGN THE PLATFORM

Scenario, architecture, boundaries and contracts — before any code.



SO FAR

- ✓ Twelve modules of techniques
- ✓ Labs 1–5 built banking services
- ✓ Each piece tested on its own

NEXT QUESTION

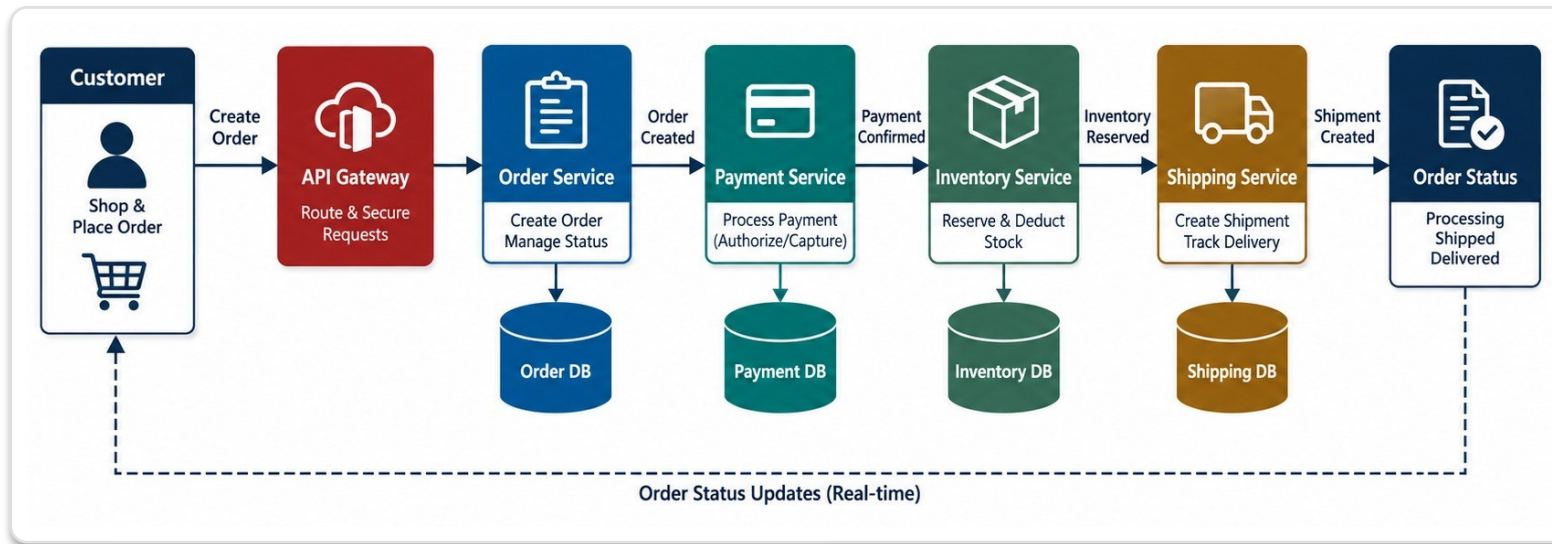
What should the platform look like before anyone writes a controller?

IN THIS PART

- 1 The business scenario
- 2 Target architecture
- 3 Boundaries — with an exercise
- 4 REST and event contracts — in code

CAPSTONE BUSINESS SCENARIO

An e-commerce order processing platform.



THE CUSTOMER STORY

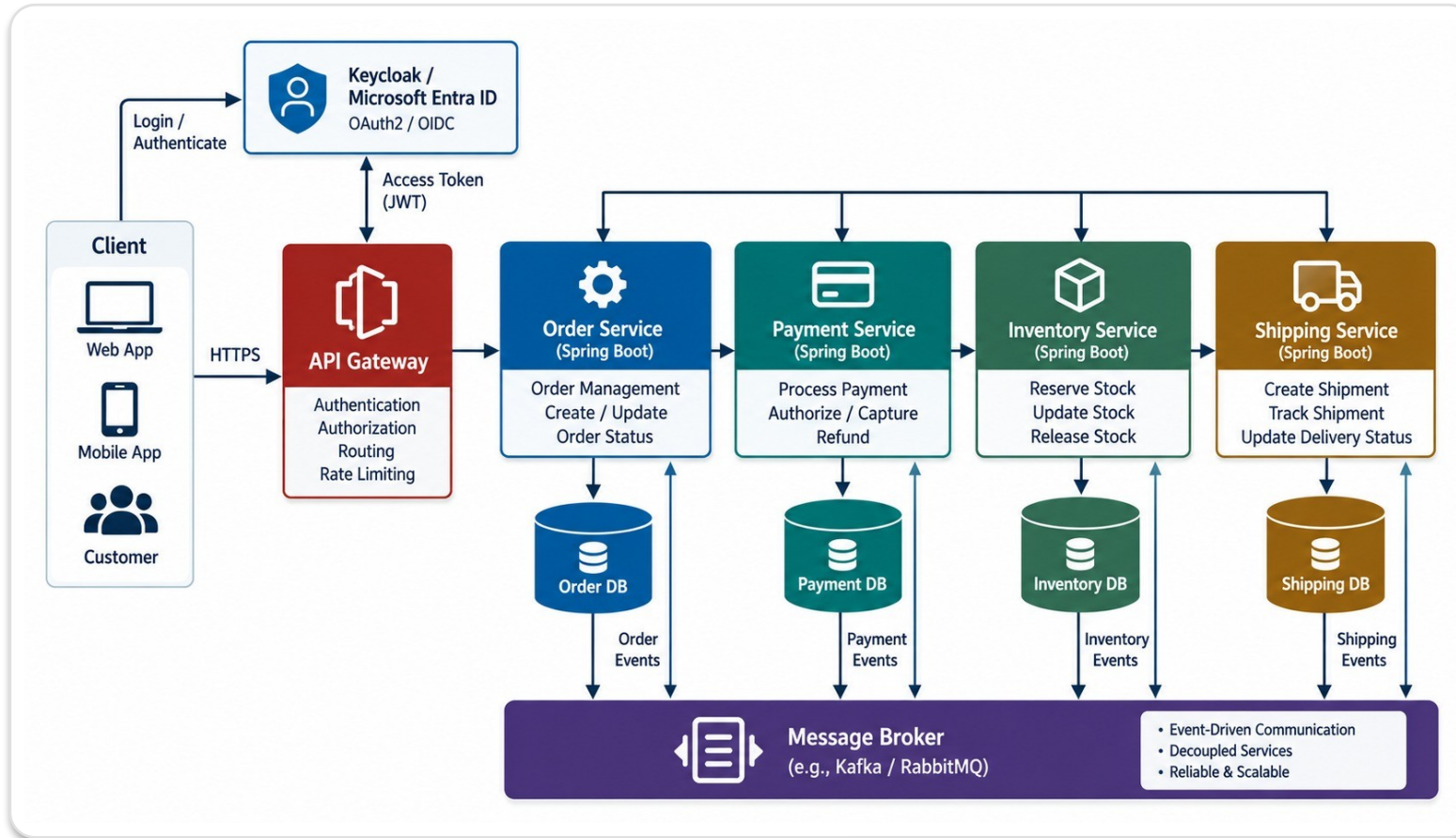
- Customer places an order
- Payment is authorized
- Stock is reserved
- The order ships
- The customer sees the status

ASK What happens when payment fails?

KEY TAKEAWAY One simple business process touches almost every topic in the course.

TARGET END-TO-END ARCHITECTURE

The complete architecture — before any implementation.



Capabilities first

Order, payment, inventory, shipping — then APIs.

Around the system

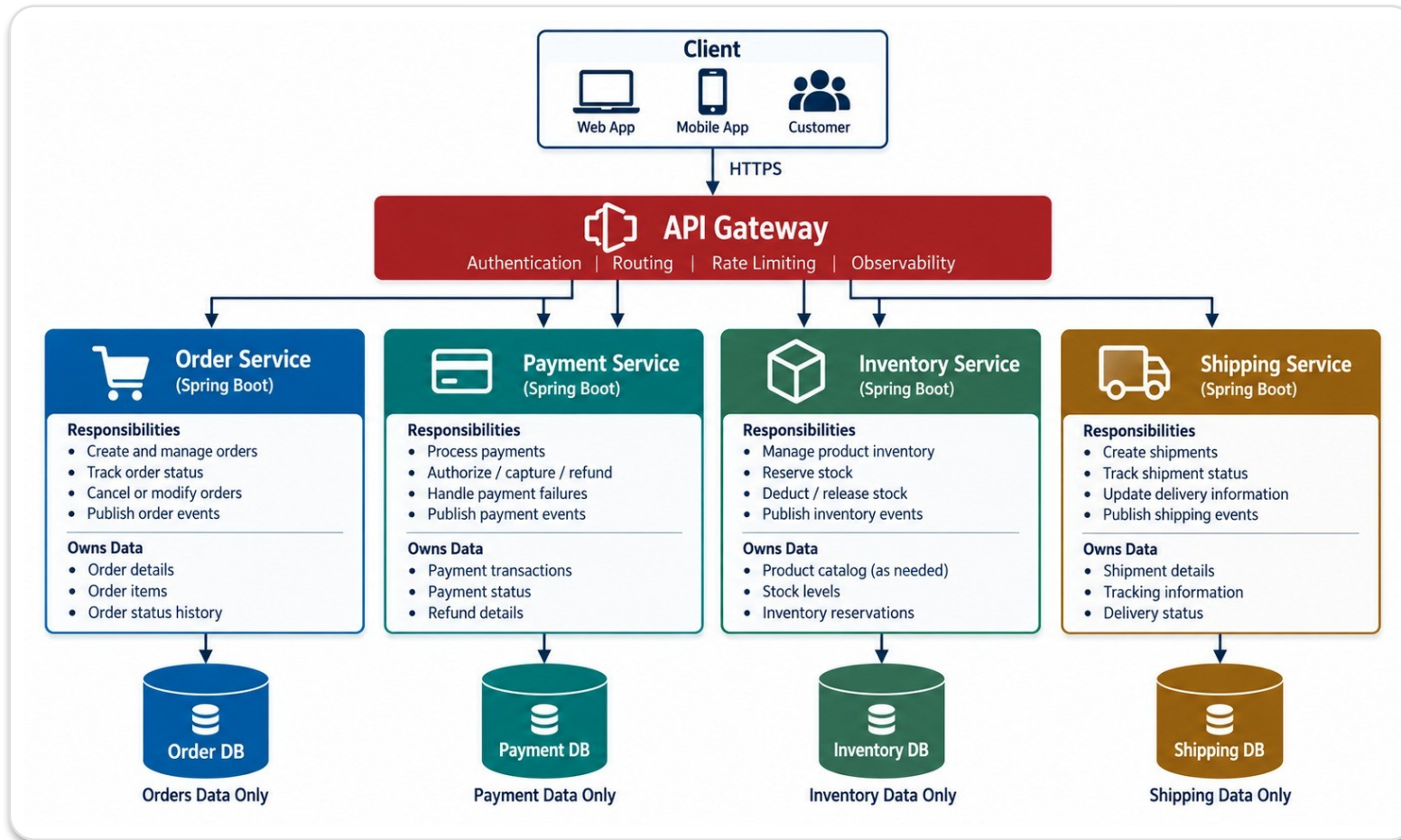
Gateway, identity, broker, pipeline, OpenShift, monitoring.

AVOID A distributed monolith: services that must deploy together.

KEY TAKEAWAY Start from capabilities and boundaries — the architecture comes before the controllers.

MICROSERVICE BOUNDARIES AND RESPONSIBILITIES

Order, Payment, Inventory and Shipping.



Order

Order lifecycle and its rules.

Payment

Authorizations, captures and refunds.

Inventory

Stock levels and reservations.

Shipping

Shipments and tracking.

KEY TAKEAWAY Clear ownership of capability and data keeps each service autonomous.

EXERCISE: DRAW THE SERVICE BOUNDARIES

Assign each capability and table to exactly one owner.

PRACTICE EXERCISE

10 MIN

SCENARIO

A team's first design has these pieces:

```
orders, order_items tables
payments, refunds tables
stock_levels, reservations tables
shipments table
Order Service reads stock_levels
Shipping updates orders.status
```

YOUR TASKS

- 1 Give every table one owning service
- 2 Find the two boundary violations
- 3 Replace each with an API call or event

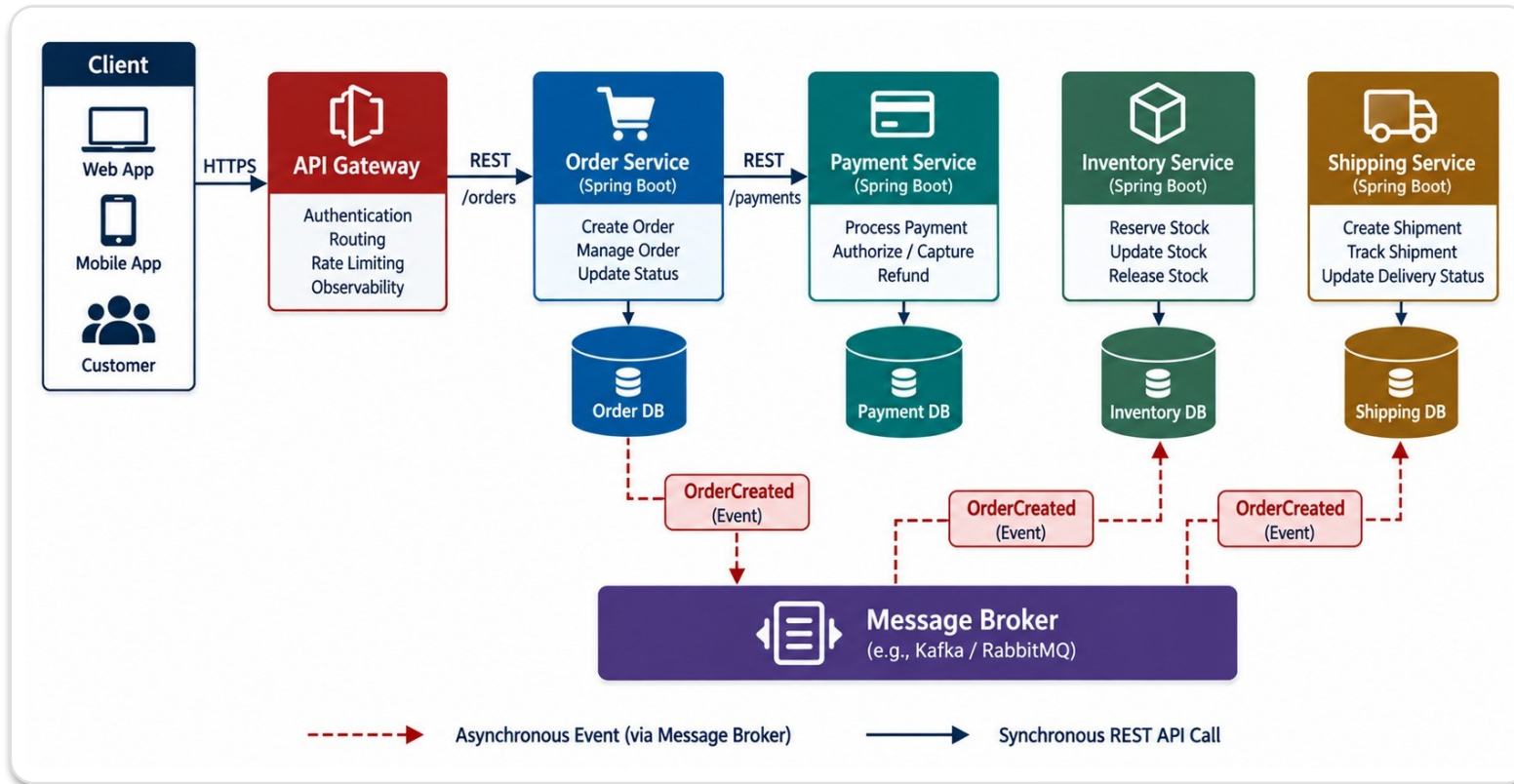
EXPECTED OUTCOME

- ✓ Order, Payment, Inventory, Shipping each own theirs
- ✓ Order → Inventory API or reservation
- ✓ Shipping publishes ShipmentDispatched
- ✓ Order Service updates its own status

KEY TAKEAWAY One owner per table — other services ask through APIs or react to events.

API AND EVENT CONTRACT DESIGN

REST endpoints and domain events.



REST — NEEDS AN ANSWER NOW

POST /orders

GET /orders/{id}

POST /payments

EVENTS — OTHERS REACT

OrderCreated

PaymentCompleted

ShipmentDispatched

KEY TAKEAWAY Design REST and event contracts deliberately — both are public APIs.

A REST CONTRACT AND AN EVENT CONTRACT

Both are versioned, documented and owned — consumers depend on them.

openapi.yaml (excerpt)

```
paths:
  /api/v1/orders:
    post:
      operationId: createOrder
      parameters:
        - in: header
          name: Idempotency-Key
          required: true
      requestBody:
        content:
          application/json:
            schema: { $ref: '#/components/schemas/CreateOrder' }
      responses:
        '201': { description: Order created }
        '400': { description: Validation failed }
        '409': { description: Duplicate request }
```

Idempotency-Key

A retried POST can't create two orders.

eventId

Consumers use it to ignore duplicates.

eventVersion

Add fields safely; never break consumers.

✓ OrderCreated v1

```
{
  "eventId": "evt-7c21",
  "eventType": "OrderCreated",
  "eventVersion": 1,
  "occurredAt": "2026-09-18T14:02:11Z",
  "correlationId": "c-41af",
  "data": {
    "orderId": "ORD-1045",
    "customerId": "CUST-0007",
    "total": { "amount": "84.50", "currency": "USD" },
    "items": [ { "sku": "SKU-221", "qty": 2 } ]
  }
}
```

KEY TAKEAWAY Version and document both contracts — and put idempotency into them from the start.

PART 2: BUILD THE SERVICES

Order Service, its integrations, its data and its failure handling.



SO FAR

- ✓ One business process drives the design
- ✓ Four services own their capabilities
- ✓ REST and event contracts are defined

NEXT QUESTION

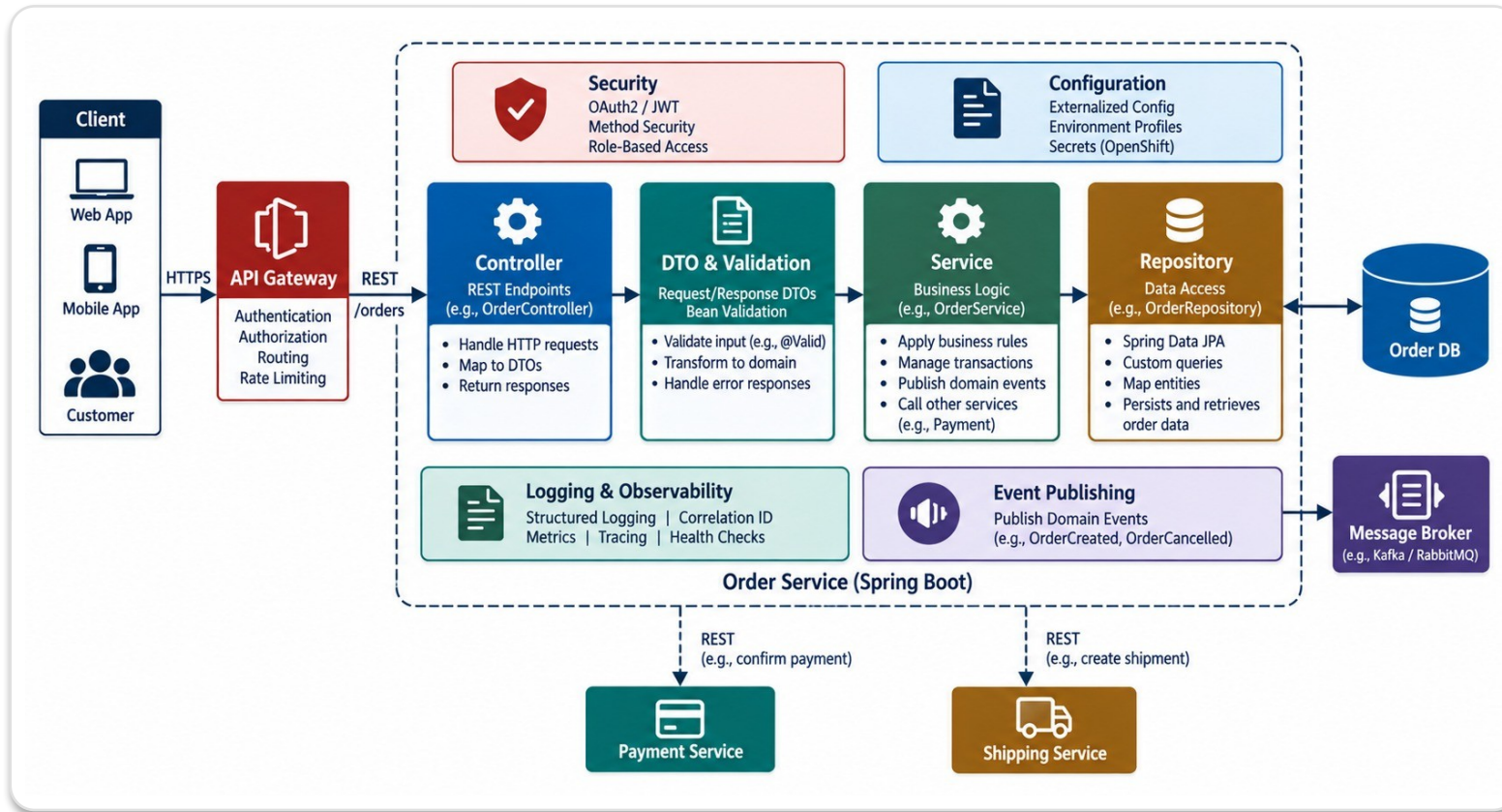
How do we build services that stay correct when their neighbours fail?

IN THIS PART

- 1 Service architecture and Order Service
- 2 Order lifecycle rules — in code
- 3 Sync and async integration
- 4 Resilience — in code
- 5 Data, failures — and an exercise

SPRING BOOT SERVICE ARCHITECTURE

Controller → service → repository → database.



Controller

HTTP, validation and DTOs — no business logic.

Service

Business rules and transactions.

Repository

Persistence only.

ACROSS EVERY LAYER

Security

Logging

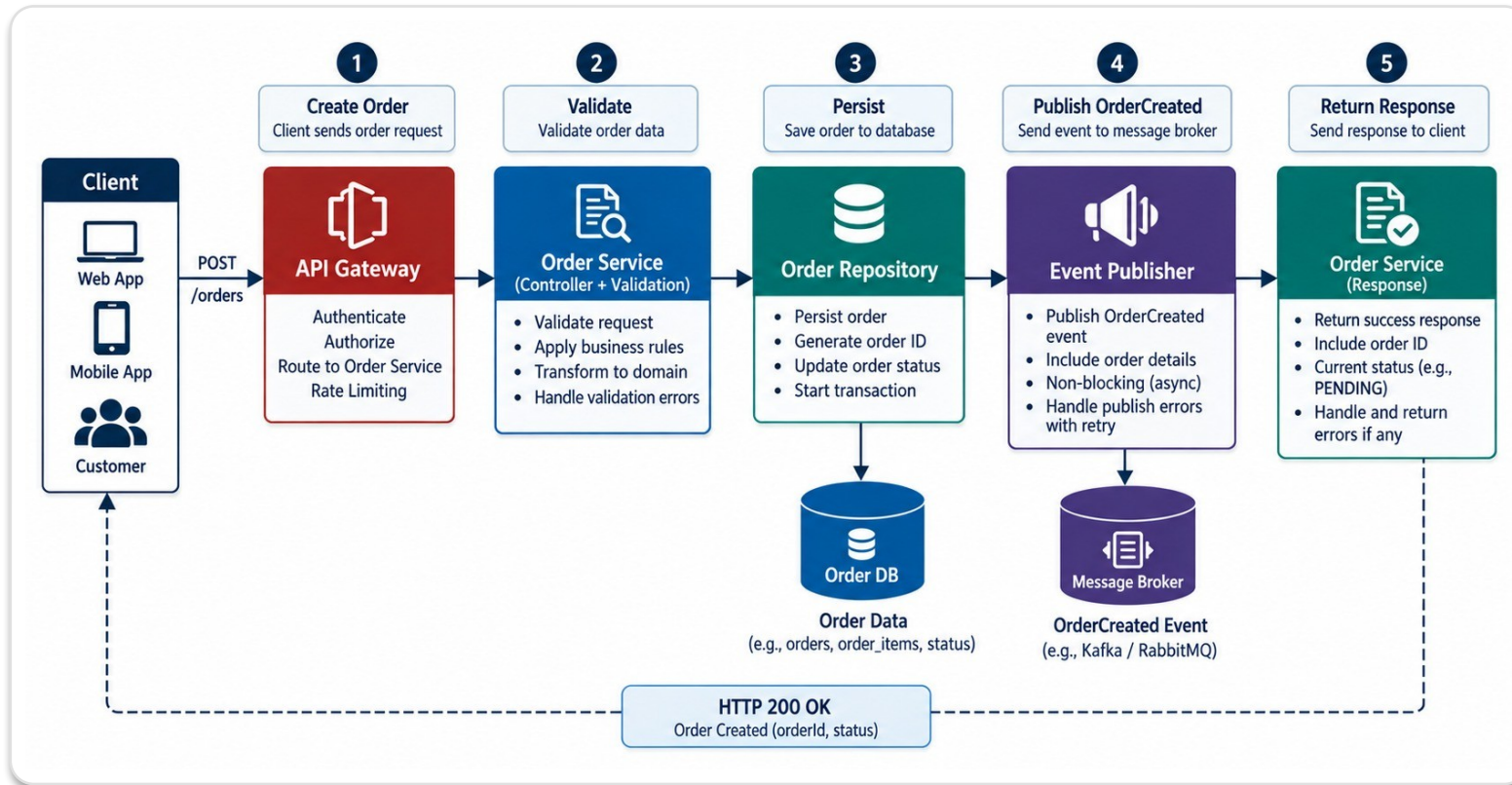
Errors

Metrics

KEY TAKEAWAY Clean layers inside every service keep code testable and maintainable.

BUILDING THE ORDER MICROSERVICE

The core service implementation.



ORDER STATES

- CREATED
- PAYMENT_PENDING
- PAID
- SHIPPED

Cancellation

Allowed before shipping; a paid order also needs a refund.

PRINCIPLE Only Order Service changes an order.

KEY TAKEAWAY Order Service owns the order lifecycle and its business rules.

ORDER LIFECYCLE RULES IN CODE

The entity guards its states; the service saves the order and its event together.

Order.java

```
public void markPaid(String paymentId) {
    requireStatus(PAYMENT_PENDING);
    this.paymentId = paymentId;
    this.status = PAID;
}

public void cancel(String reason) {
    if (status == SHIPPED || status == CANCELLED)
        throw new InvalidOrderStateException(id, status);
    this.refundRequired = (status == PAID);
    this.status = CANCELLED;
    this.cancelReason = reason;
}
```

Guarded states

Illegal transitions fail loudly.

✓ OrderService.java

```
@Transactional
public OrderResponse create(CreateOrder cmd, String idempotencyKey) {
    return orders.findByIdempotencyKey(idempotencyKey)
        .map(OrderResponse::from) // retried request
        .orElseGet(() -> {
            Order order = orders.save(Order.create(cmd, idempotencyKey));
            outbox.save(OutboxEvent.orderCreated(order)); // same transaction
            return OrderResponse.from(order);
        });

    // a relay publishes outbox rows to the broker
}
```

Idempotent create

A retried request returns the same order.

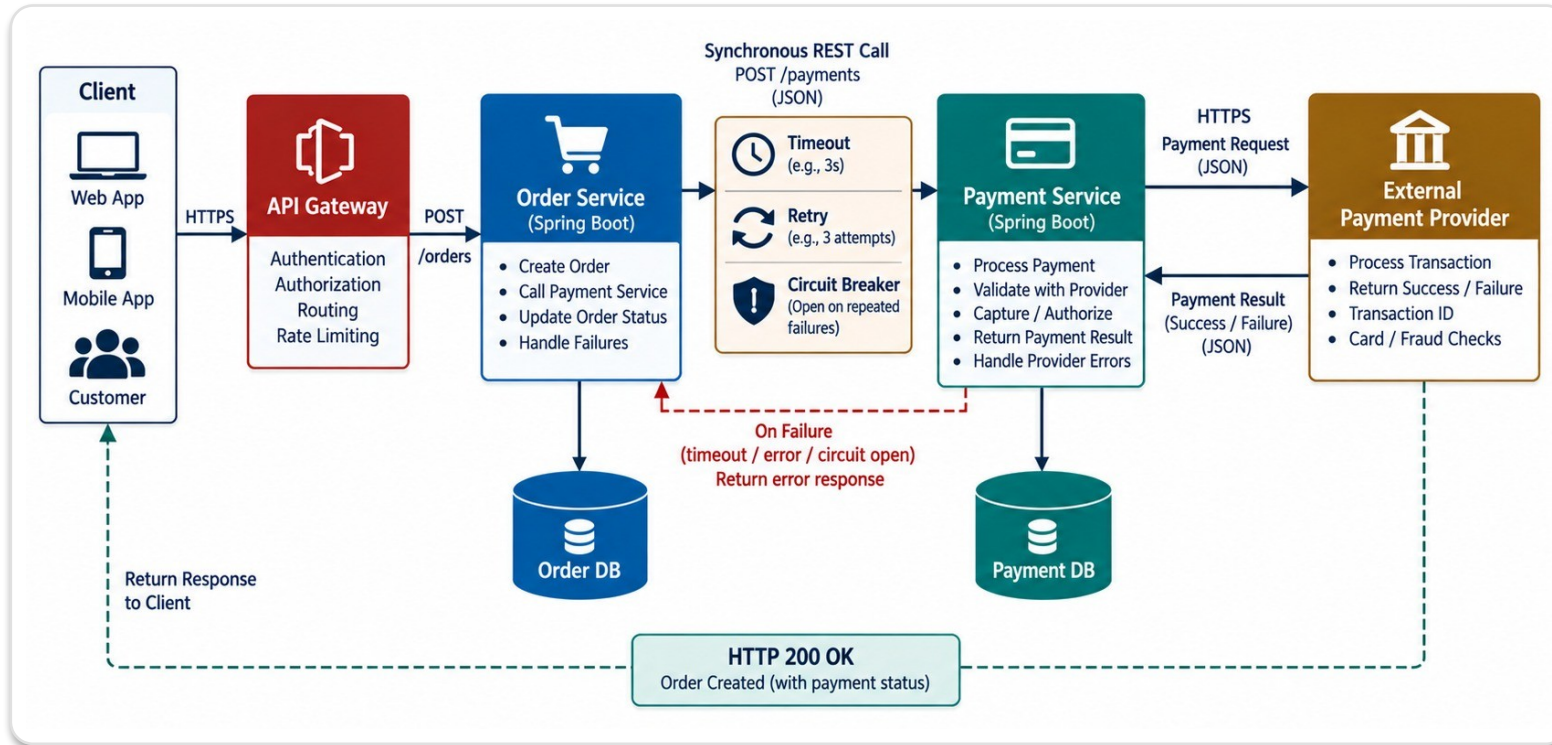
Outbox

The order and its event commit together.

KEY TAKEAWAY Put state rules in the entity, and save each change with its event in one transaction.

SYNCHRONOUS SERVICE INTEGRATION

Order → Payment REST call.



POSSIBLE OUTCOMES

- ? Authorized
- ? Declined
- ? Timed out
- ? Unavailable

Never wait forever

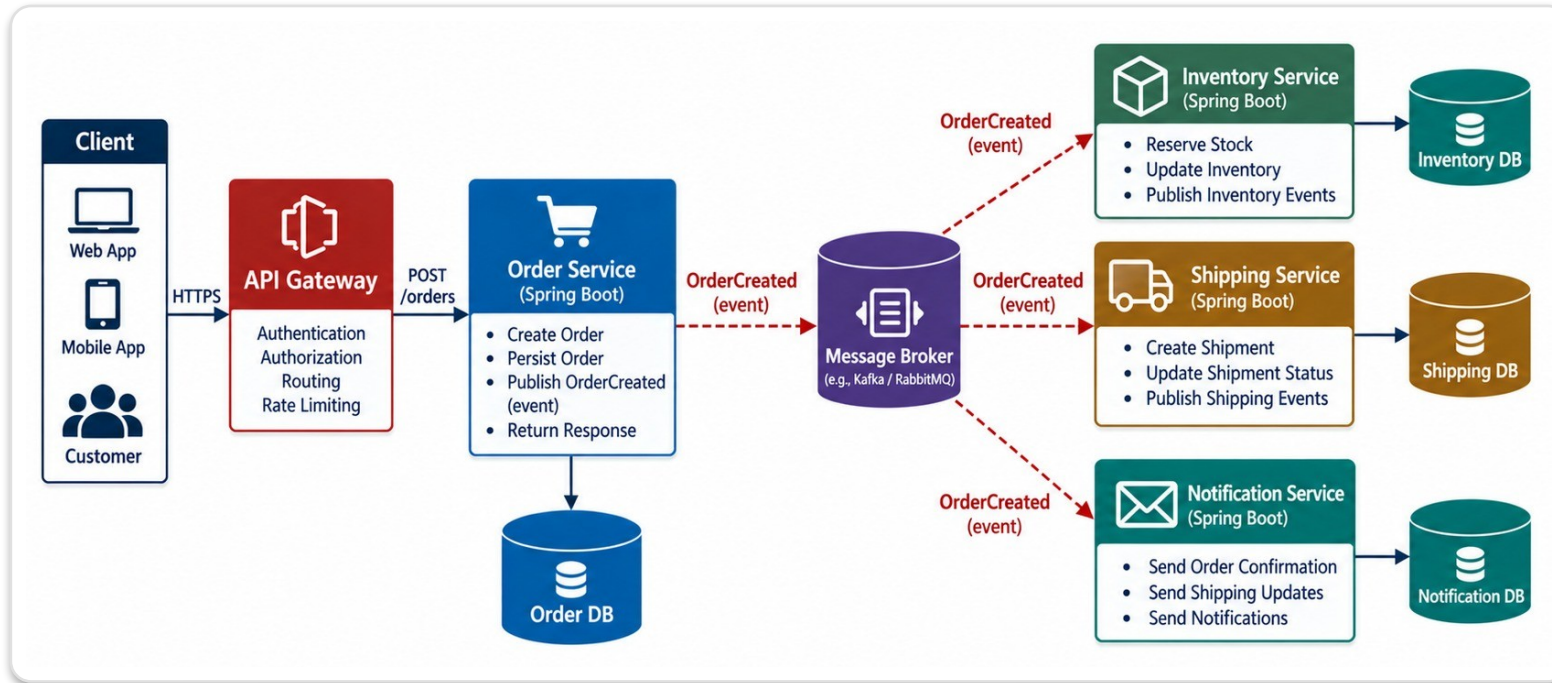
Connect and read timeouts on every call.

TIMED OUT ≠ FAILED The payment may have succeeded.

KEY TAKEAWAY Synchronous calls need timeouts, safe retries and circuit breakers.

ASYNCHRONOUS EVENT INTEGRATION

OrderCreated → broker → *Inventory and Shipping*.



Gives

Loose coupling, independent scaling, room for new consumers.

Costs

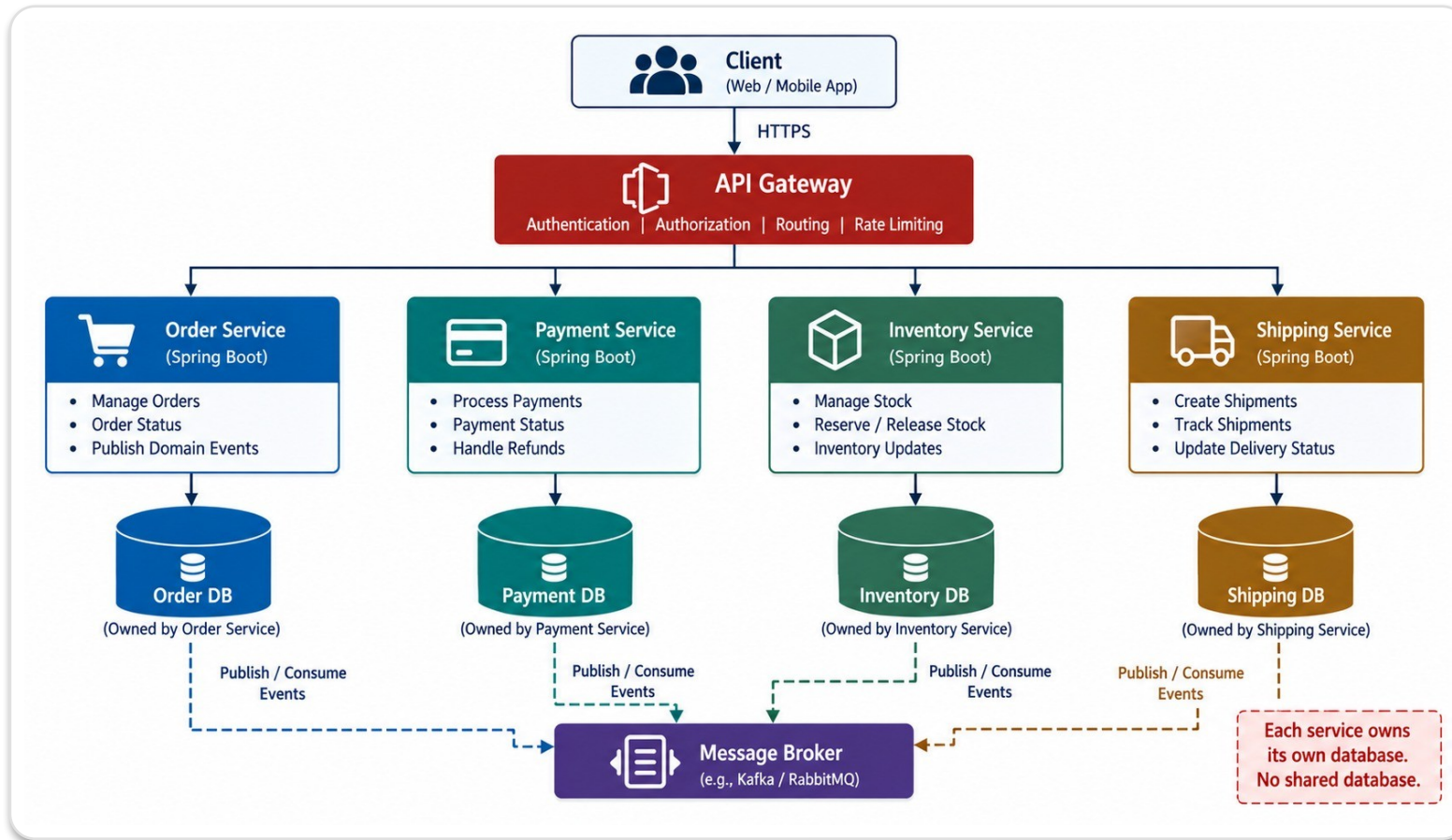
Eventual consistency, duplicates and harder tracing.

CONSUMERS must be idempotent.

KEY TAKEAWAY Events decouple services — so consumers must handle duplicates and eventual consistency.

DATA PERSISTENCE AND SERVICE-OWNED DATABASES

JPA/SQL and database-per-service.



Tempting shortcut

Order Service joins the inventory tables directly.

Service-owned data

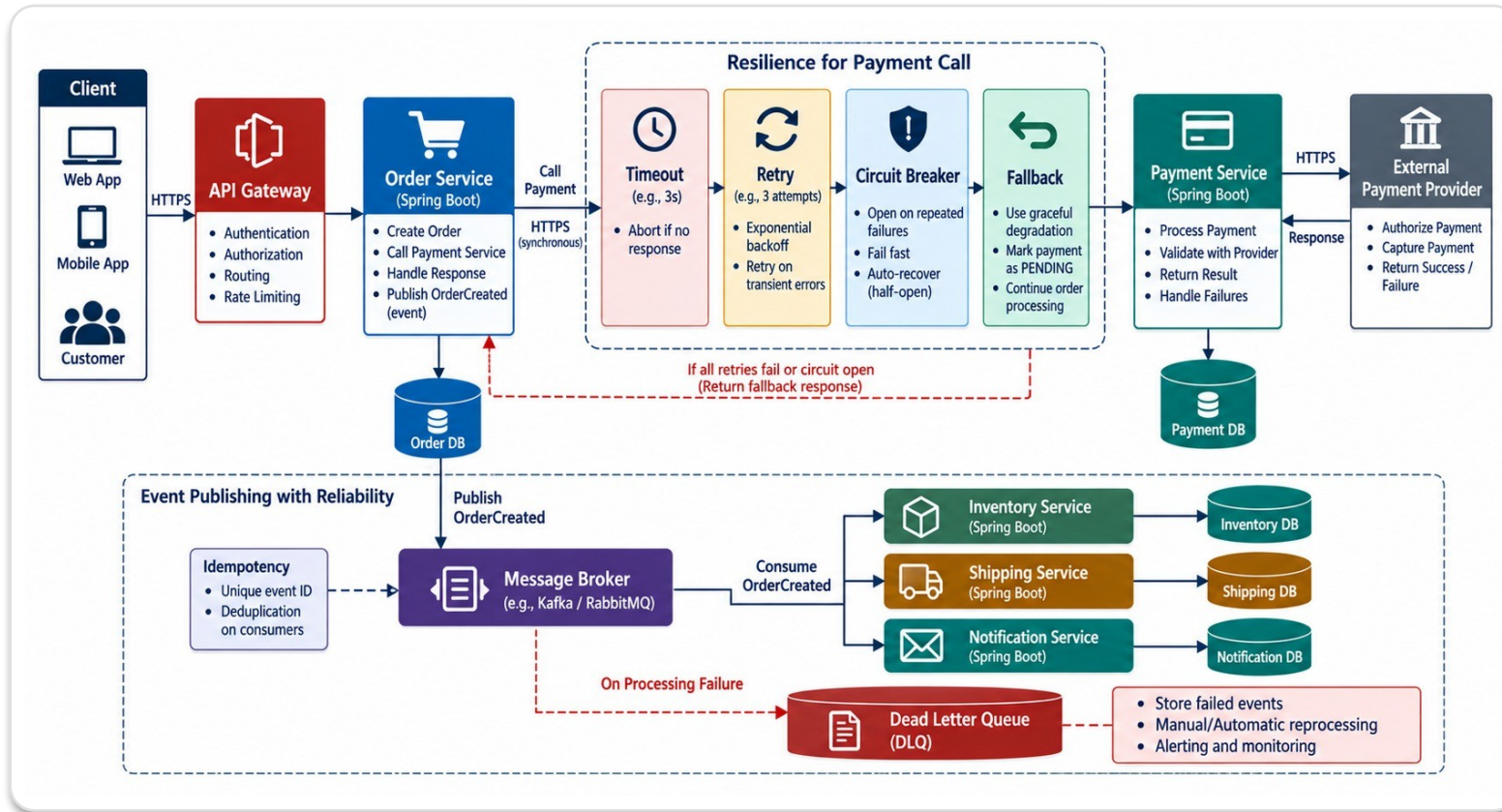
Ask the Inventory API, or keep a local copy from events.

TRANSACTIONS Local per service; sagas across services.

KEY TAKEAWAY Each service owns its database — share data through APIs and events, never tables.

RESILIENCE AND FAILURE HANDLING

Timeout, retry, circuit breaker and idempotency.



THE TOOLKIT

Timeout

Retry + backoff

Circuit breaker

Idempotency

Dead-letter queue

Bulkhead

RULE Only retry what's safe to repeat.

KEY TAKEAWAY Design for failure: bound waits, retry carefully, fail fast and isolate bad messages.

RESILIENT CALLS AND IDEMPOTENT CONSUMERS

Bound the synchronous call; make the event consumer safe to run twice.

PaymentClient.java

```
@CircuitBreaker(name = "payment", fallbackMethod = "pending")
@Retry(name = "payment")
public PaymentResult authorize(Order order) {
    return paymentApi.post()
        .uri("/api/v1/payments")
        .header("Idempotency-Key", "pay-" + order.id())
        .body(PaymentRequest.from(order))
        .retrieve()
        .body(PaymentResult.class);
}

PaymentResult pending(Order order, Throwable cause) {
    return PaymentResult.unknown(order.id()); // never "approved"
}
```

Same key on retry

Payment Service charges once.

Fallback

Unknown, then reconcile — never invent success.

Processed events

A redelivery changes nothing.

✓ InventoryListener.java

```
@KafkaListener(topics = "orders.events", groupId = "inventory-service")
@Transactional
public void on(OrderCreated event) {
    if (processed.existsById(event.eventId())) {
        log.info("Duplicate ignored eventId={}", event.eventId());
        return;
    }
    reservations.reserve(event.data().orderId(), event.data().items());
    processed.save(new ProcessedEvent(event.eventId()));
}

# resilience4j.timelimiter.instances.payment.timeout-duration: 2s
# resilience4j.retry.instances.payment.max-attempts: 3
```

KEY TAKEAWAY Retry only with idempotency, fall back to "unknown" — never to "approved".

EXERCISE: TRACE A FAILED PAYMENT

Decide what the platform does at each step of a bad afternoon.

PRACTICE EXERCISE

12 MIN

SCENARIO

ORD-1045 is created. Then:

- 1 Payment call times out after 2s
- 2 Retry succeeds – or did it charge twice?
- 3 Payment Service goes down for 5 minutes
- 4 OrderCreated is delivered twice
- 5 A poison message keeps failing

YOUR TASKS

- 1 Give the order's state after each step
- 2 Name the control that protects it
- 3 Say what the customer sees

EXPECTED OUTCOME

- ✓ 1 PAYMENT_PENDING — timeout
- ✓ 2 One charge — same Idempotency-Key
- ✓ 3 Circuit opens; status unknown
- ✓ 4 One reservation — processed events
- ✓ 5 Dead-letter queue + alert

KEY TAKEAWAY Every failure has a designed outcome — and none of them is a made-up success.

PART 3: PROTECT AND TEST

Security, configuration and tests — built in, not bolted on.



SO FAR

- ✓ Clean layers and a guarded lifecycle
- ✓ Calls bounded; consumers idempotent
- ✓ Each service owns its data

NEXT QUESTION

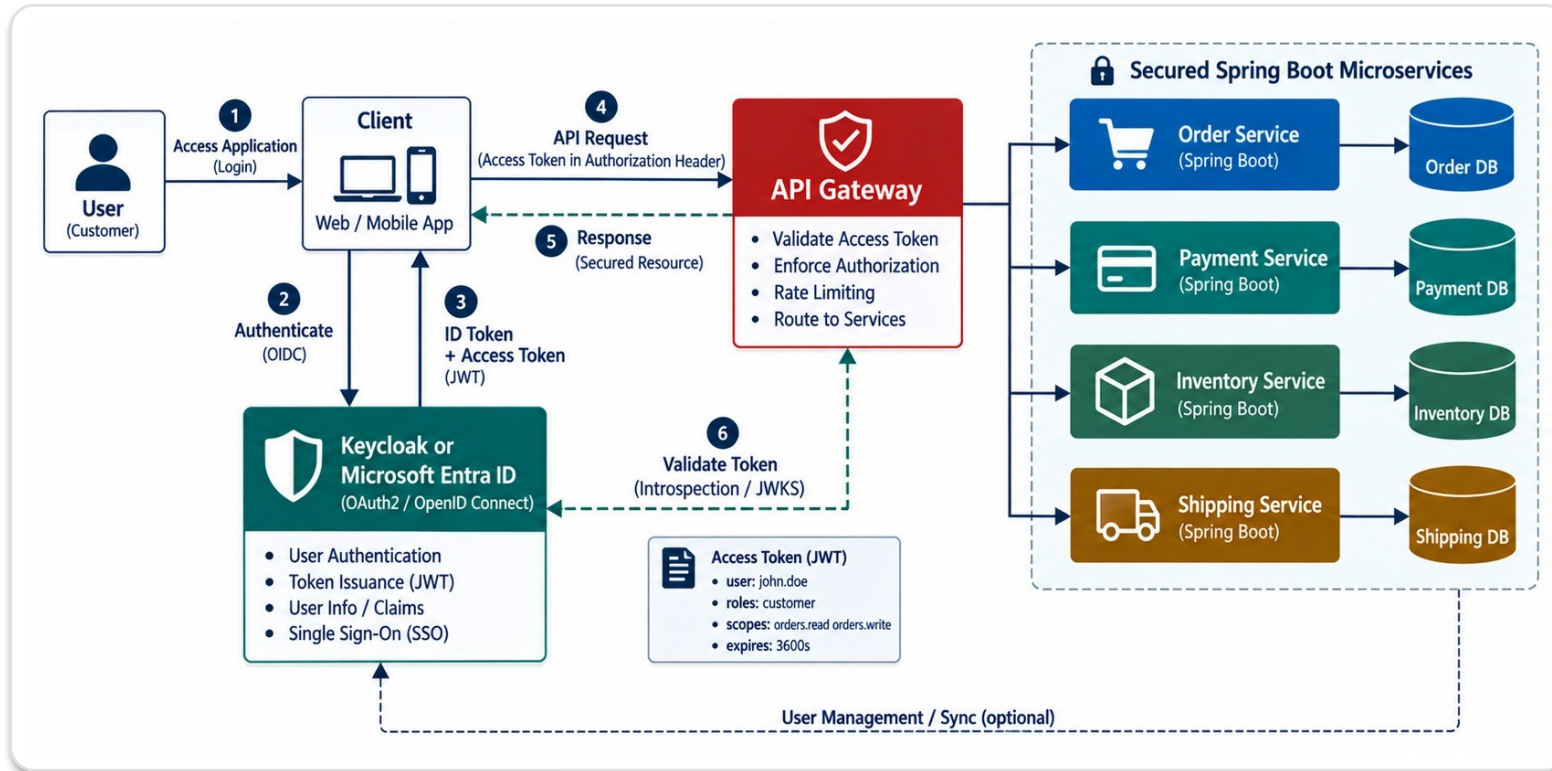
How do we know the platform is secure — and still correct after every change?

IN THIS PART

- 1 API security and authentication
- 2 Secrets and configuration
- 3 Automated testing strategy

API SECURITY AND AUTHENTICATION

OAuth2/OIDC with Keycloak or Microsoft Entra ID.



Authenticate

Every service validates issuer, audience and expiry.

Authorize

Scopes per endpoint — orders.write, payments.authorize.

Resource checks

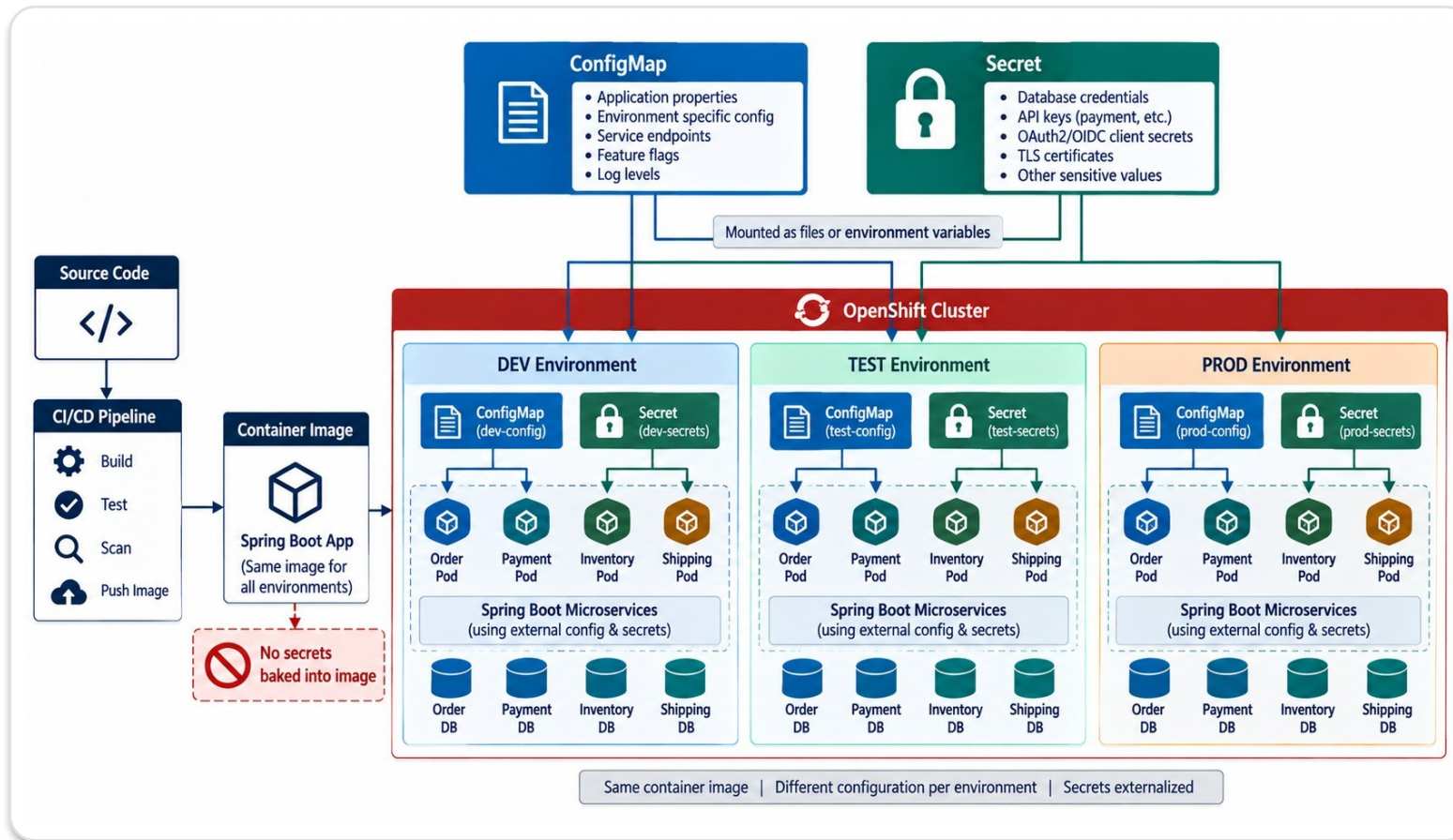
This customer may see only their orders.

LAB 3 HMAC JWTs from issue-jwt.py.

KEY TAKEAWAY Validate tokens and authorize each resource — in every service, from day one.

SECRETS AND CONFIGURATION MANAGEMENT

External configuration, OpenShift Secrets and profiles.



ConfigMap

URLs, topics, feature flags.

Secret

DB passwords, client secrets, API keys.

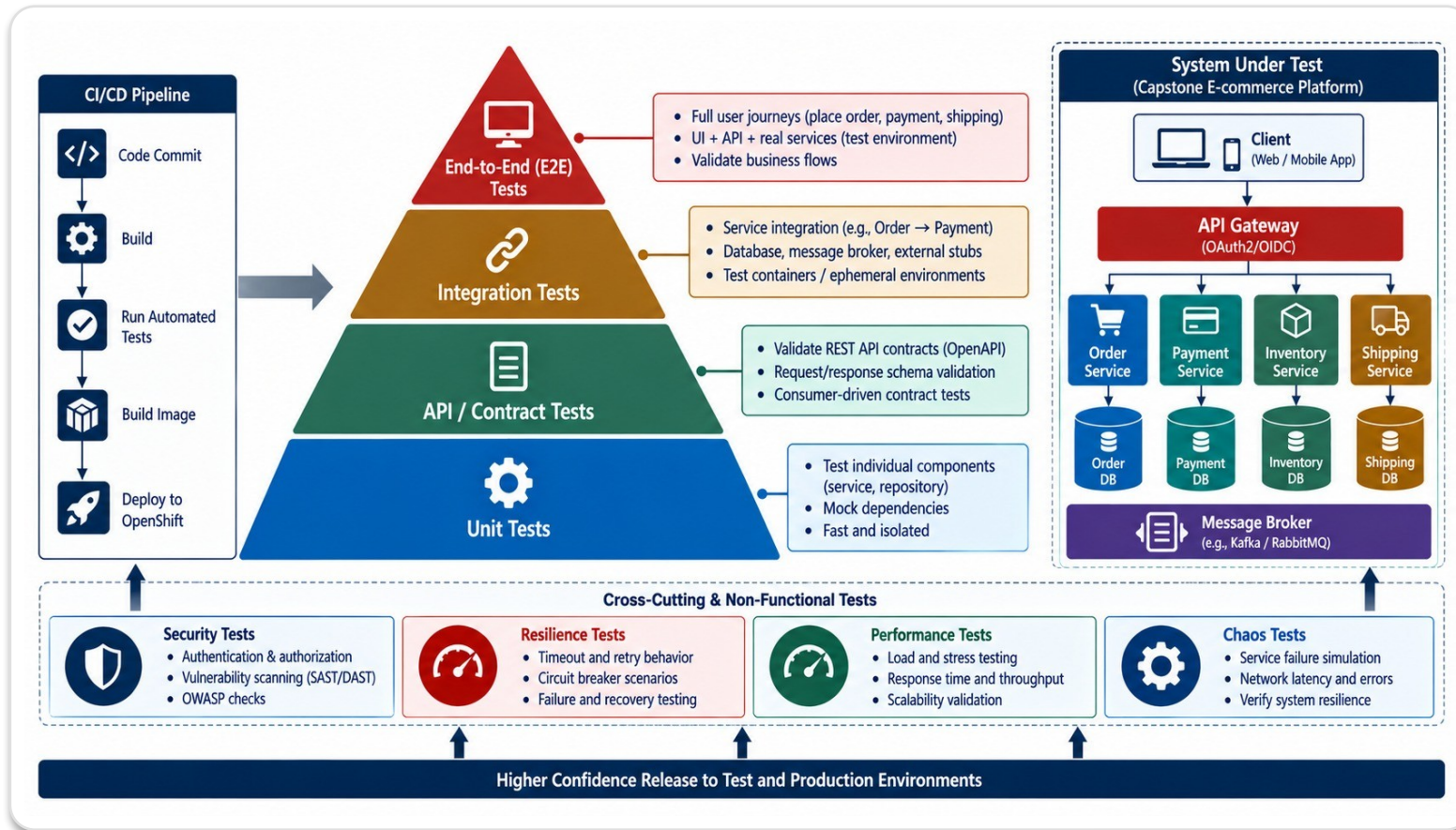
NEVER

- ✗ Secrets in Git or images
- ✗ Rebuilding per environment

KEY TAKEAWAY One image, many environments — configuration and secrets arrive at runtime.

AUTOMATED TESTING STRATEGY

Unit, API, integration, contract and resilience tests.



Unit

Order state rules and policy logic.

Integration

Testcontainers with real Postgres and Kafka.

Contract

Consumers and providers agree on APIs and events.

Security and resilience

401/403 cases, timeouts, duplicates.

KEY TAKEAWAY Automate tests at every layer so problems surface before deployment.

PART 4: DELIVER TO OPENSIFT

From a pull request to running, scalable pods.



SO FAR

- ✓ Tokens validated in every service
- ✓ Secrets arrive at runtime
- ✓ Tests cover rules, data and failures

NEXT QUESTION

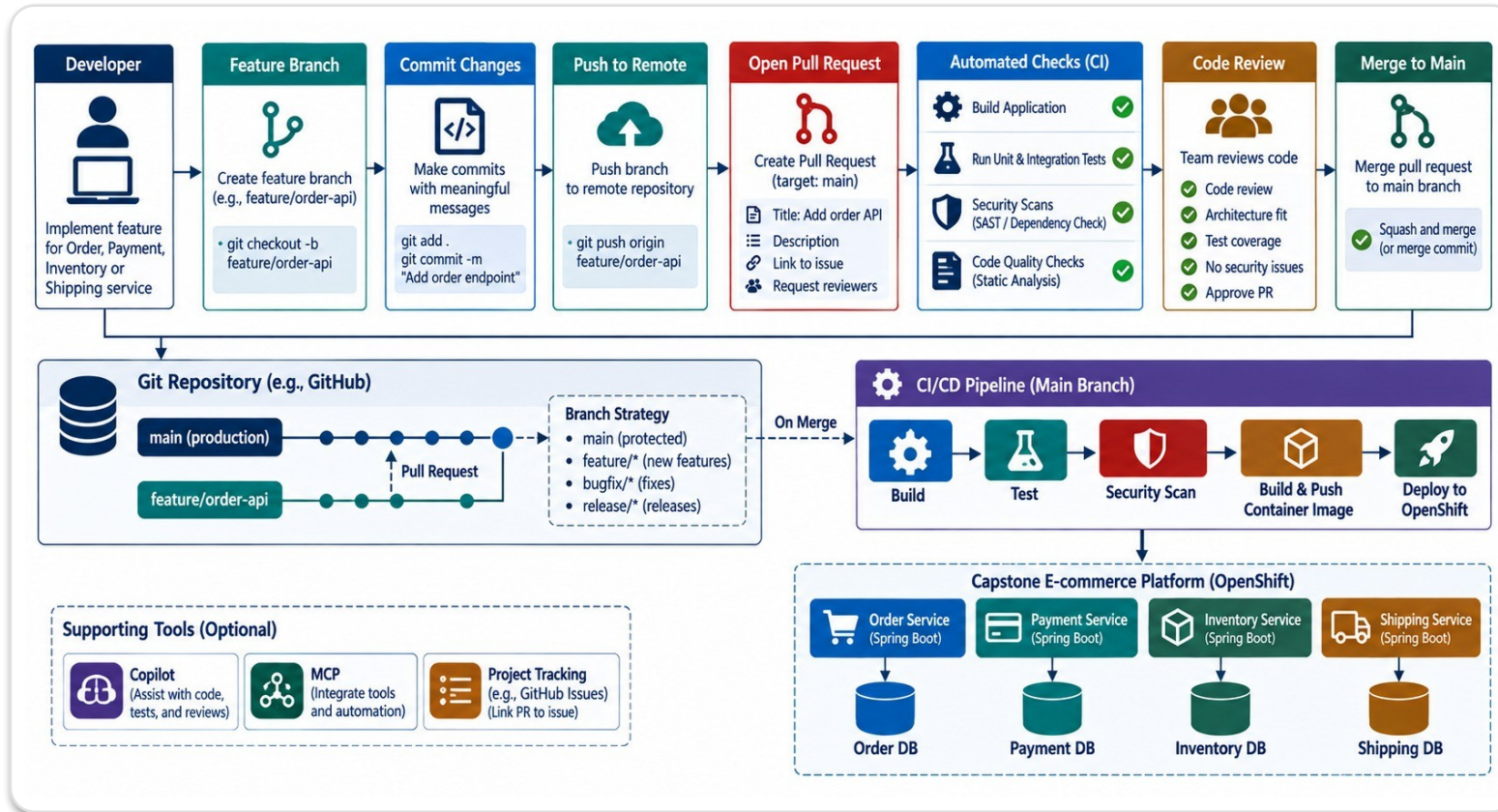
How does a reviewed change reach production safely — and roll back if it doesn't?

IN THIS PART

- 1 Git workflow and PR gates
- 2 CI/CD — in code
- 3 Containers and OpenShift
- 4 Scaling, health and rollouts
- 5 Exercise: missing risk assessments

GIT WORKFLOW AND PULL REQUEST QUALITY GATES

Branching, review and automated checks.



BEFORE MERGING

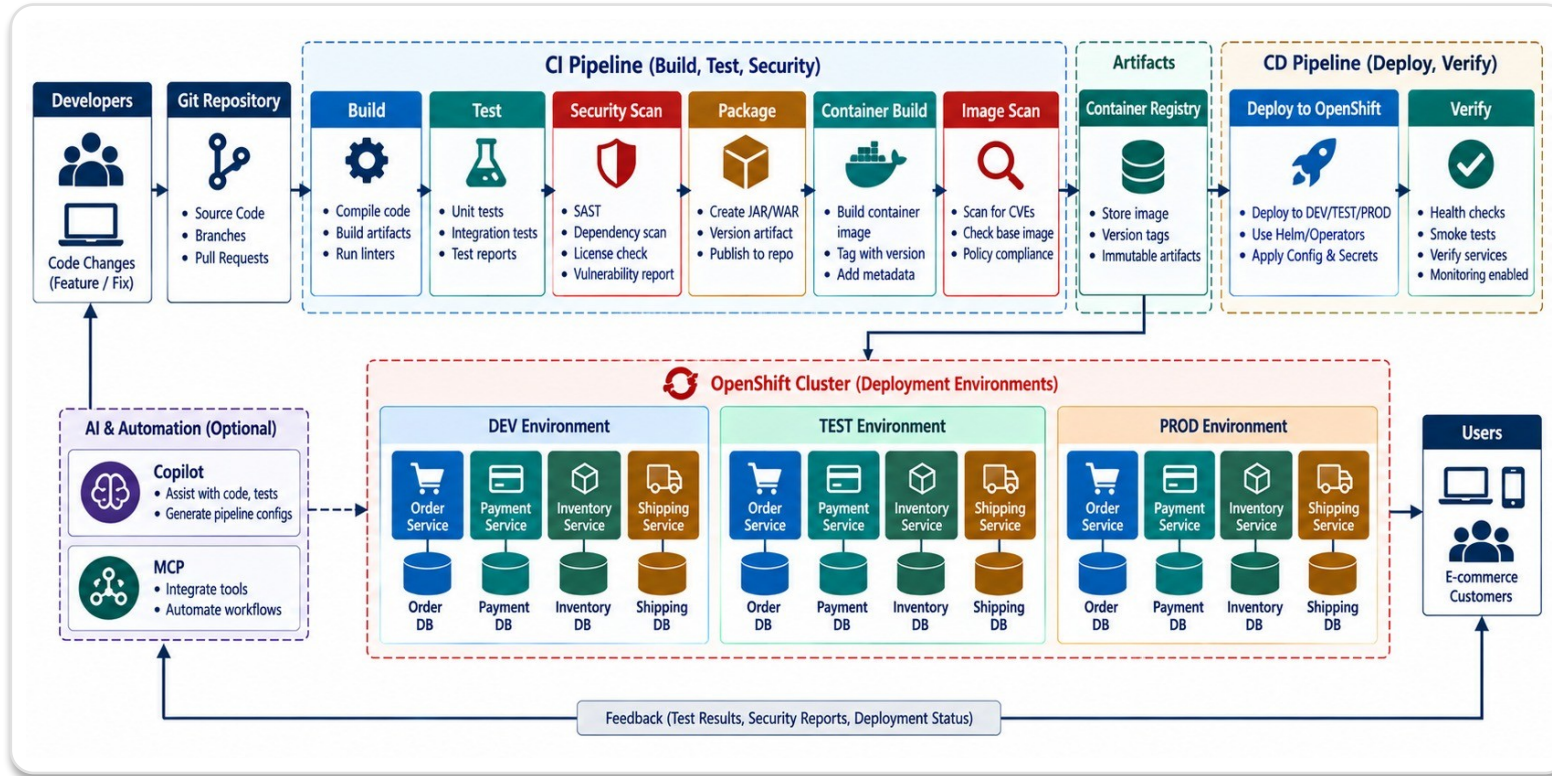
- ✓ Build and tests pass
- ✓ Scans clean
- ✓ One approving review
- ✓ Contracts unchanged or versioned

AI CODE TOO Copilot changes pass the same gates.

KEY TAKEAWAY Every change — human or AI-written — passes the same automated and human gates.

CI/CD PIPELINE INTEGRATION

GitHub Actions / Jenkins → build → test → scan → image → deploy.



Stop conditions

Failing tests, critical vulnerabilities, leaked secrets.

Build once

The same image digest moves from dev to prod.

VENDOR matters less than the gates.

KEY TAKEAWAY Automate the path from commit to deployment — and promote one versioned artifact.

THE CAPSTONE PIPELINE IN ONE VIEW

Test, build one image, deploy it by digest — and know how to undo it.

`.github/workflows/order-service.yml`

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '21' }
      - run: mvn -B verify          # unit + Testcontainers
      - uses: aquasecurity/trivy-action@0.28.0
        with: { scan-type: fs, severity: CRITICAL, exit-code: '1' }
      - run: |
          podman build -t $IMAGE:${{ github.sha }} .
          podman push --digestfile digest $IMAGE:${{ github.sha }}
      - run: |
          oc set image deploy/order-service \
            order-service=$IMAGE@$(cat digest)
          oc rollout status deploy/order-service --timeout=180s
```

✖ When the rollout goes wrong

```
oc rollout status deploy/order-service
# error: deployment exceeded its progress deadline

oc get pods -l app=order-service
oc logs deploy/order-service --tail=50

oc rollout undo deploy/order-service
oc rollout status deploy/order-service

# then: fix forward through a new PR
```

Commit SHA tag

Traceable back to the change.

Deploy by digest

Exactly the image that was scanned.

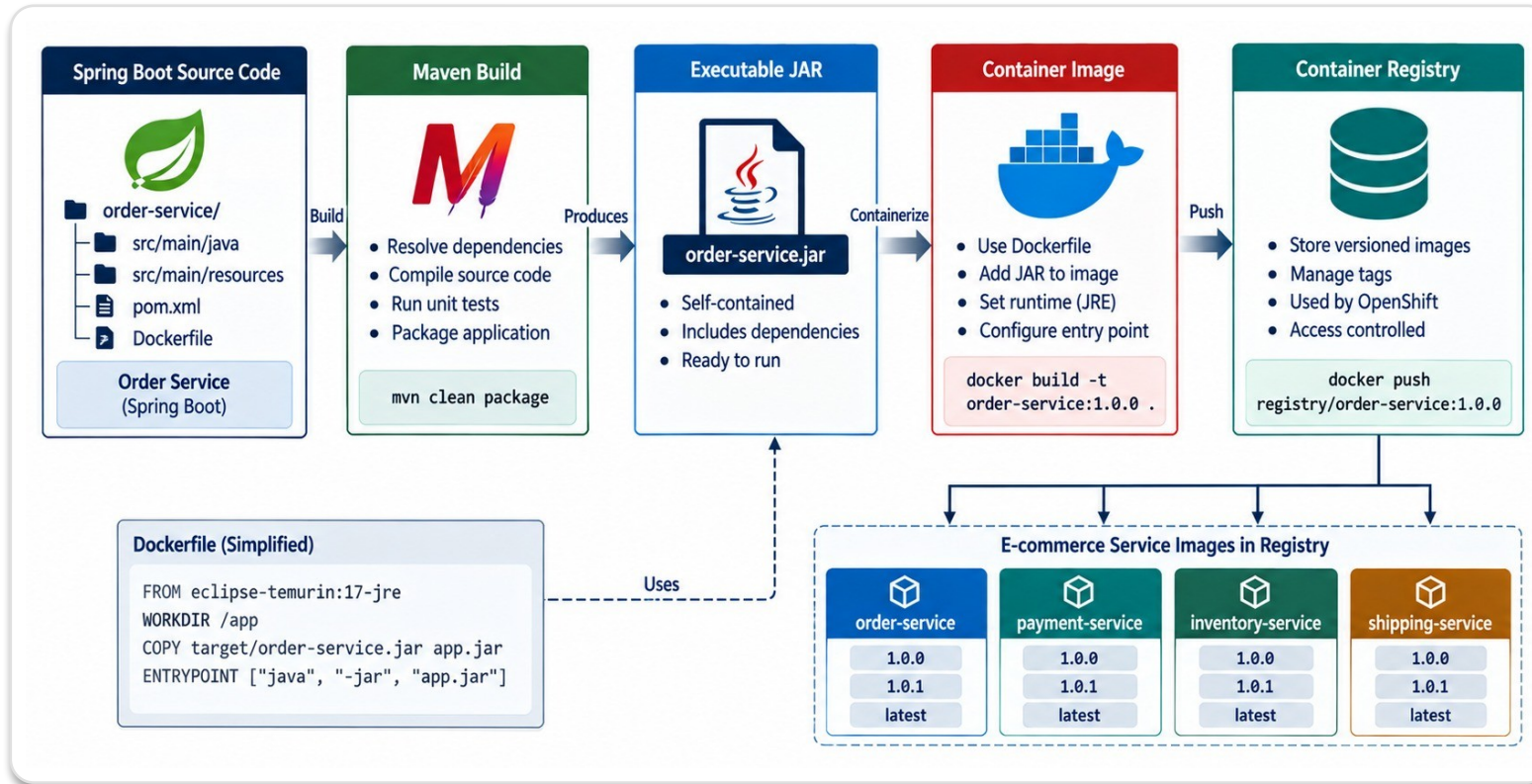
Rollback

Practised — Lab 4's oc rollout undo.

KEY TAKEAWAY Gate on tests and scans, deploy one image by digest, and practise the rollback.

CONTAINERIZING THE MICROSERVICES

Docker/OCI images, tagging and the registry.



EVERY IMAGE

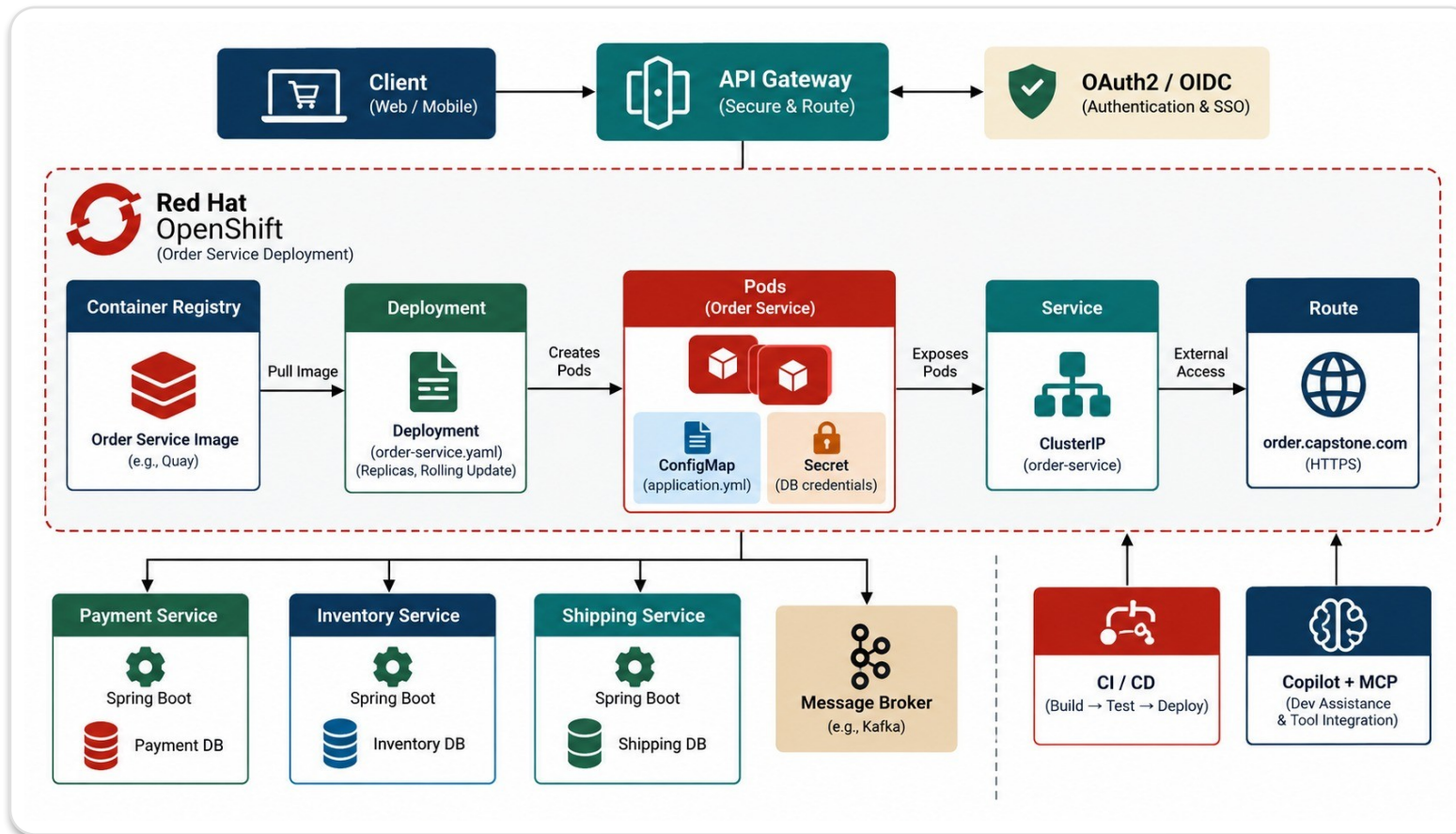
- ✓ Multi-stage build
- ✓ Non-root user
- ✓ Versioned tag, no :latest
- ✓ Scanned before push
- ✓ No secrets inside

IMAGE VS CONTAINER Template vs running instance.

KEY TAKEAWAY Every service ships as a versioned, scanned, secret-free container image.

DEPLOYING TO OPENSHIFT

Deployment, Service, Route, ConfigMap and Secret.



Deployment

Runs the pods with probes and resources.

Service

Stable internal address for other services.

Route

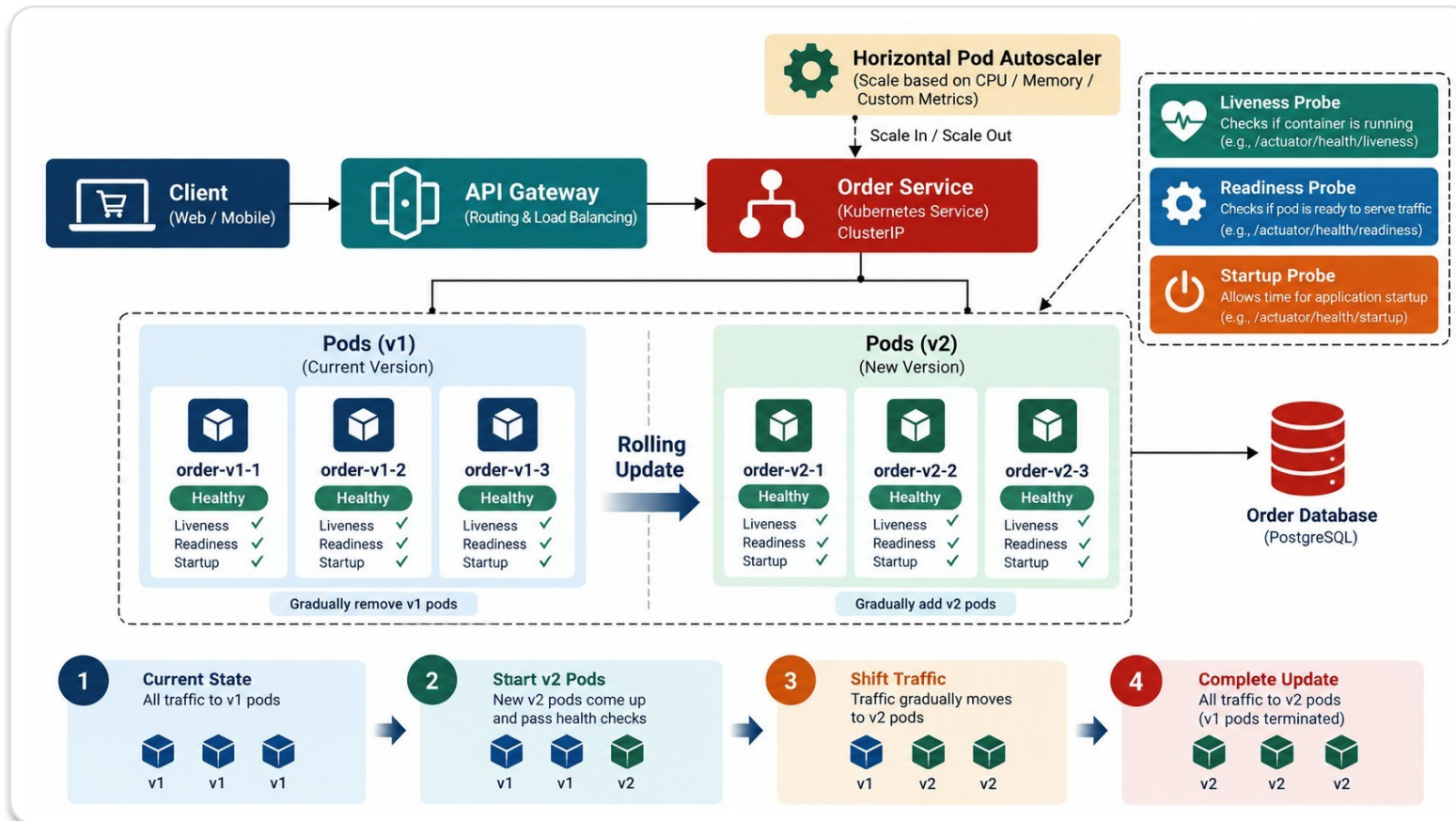
External HTTPS — only for public entry points.

INTERNAL ONLY Payment and Inventory need no Route.

KEY TAKEAWAY OpenShift runs each service as Deployment → Pods → Service, exposing only what must be public.

SCALING, HEALTH CHECKS, AND ROLLOUTS

Readiness and liveness, replicas and rolling updates.



Readiness

No traffic until the pod can serve.

Liveness

Restart a stuck pod.

Rolling update

New pods ready before old ones stop.

RECOVERY oc rollout undo restores the last version.

KEY TAKEAWAY Replicas add capacity, probes guard traffic, rolling updates replace versions safely.

EXERCISE: WHY ARE RISK ASSESSMENTS MISSING?

Troubleshoot the deployed banking platform in a sensible order.

PRACTICE EXERCISE

10 MIN

SCENARIO

On OpenShift, transactions are submitted successfully, but GET `/api/v1/assessments/{transactionId}` returns 404 for every new one.

YOUR TASKS

- 1 Choose which service to check first
- 2 List the checks in order
- 3 Name the evidence that confirms each

EXPECTED OUTCOME

- ✓ Was TransactionSubmitted published?
- ✓ Broker reachable; consumer group risk-assessment-service
- ✓ Risk pod Ready; logs show errors?
- ✓ ConfigMap topic and Secret values set
- ✓ 401/403 on the lookup call?

KEY TAKEAWAY Follow the event: published, delivered, consumed, stored — then check config and access.

PART 5: OPERATE, ACCELERATE AND DEMONSTRATE

Observability, AI working through the platform, and the final demo.



SO FAR

- ✓ Every change passes PR and pipeline gates
- ✓ One image deployed by digest
- ✓ Probes, rollouts and rollback in place

NEXT QUESTION

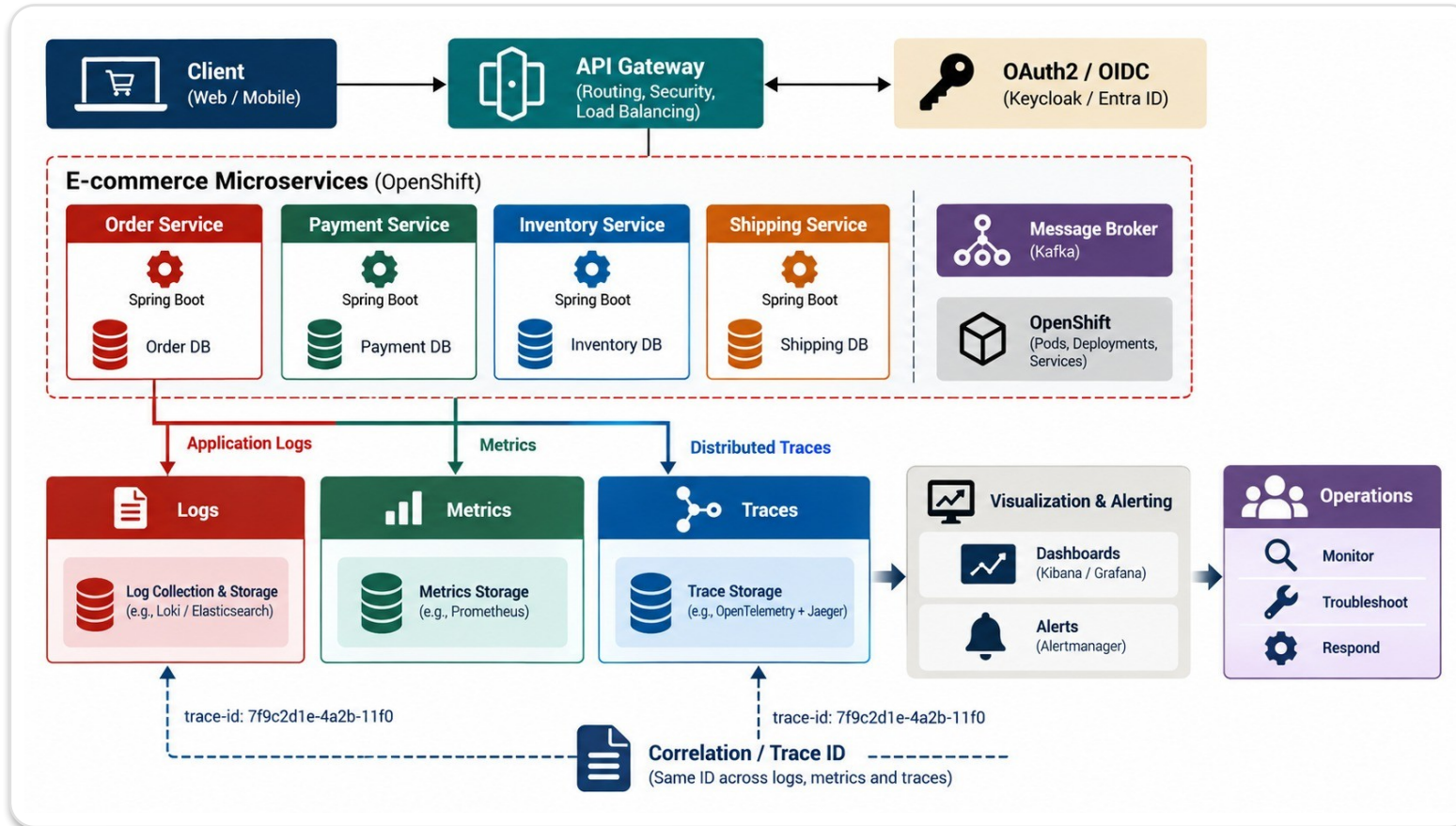
Can we see, explain and prove how the platform behaves — including when it fails?

IN THIS PART

- 1 Observability and monitoring
- 2 Copilot and agentic AI with MCP
- 3 The demo — in commands
- 4 Official capstone demonstration
- 5 Readiness review and the full architecture

OBSERVABILITY AND MONITORING

Logs, metrics, traces, dashboards and alerts.



Metrics find it

Error rate on POST /orders jumps.

Traces locate it

The payment span takes 2 seconds.

Logs explain it

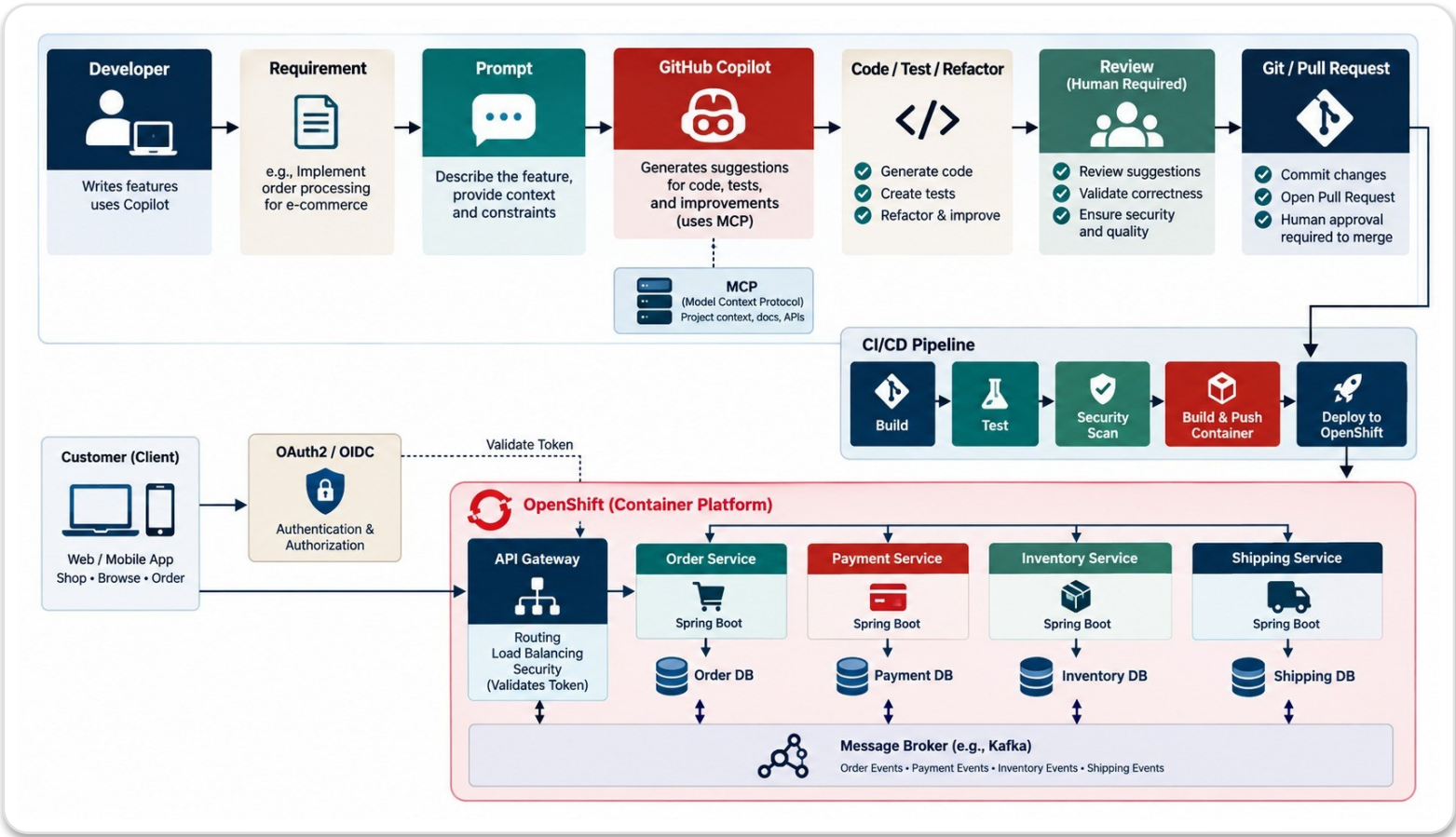
Timeouts, with the correlation ID.

ALERT ON HOLD backlog and DLQ depth too.

KEY TAKEAWAY Follow one request end to end — metrics find it, traces locate it, logs explain it.

AI-ASSISTED DEVELOPMENT WITH GITHUB COPILOT

Generate, test, refactor and troubleshoot.



GOOD USES

- Draft code
- Write tests
- Refactor
- Explain errors

After it generates

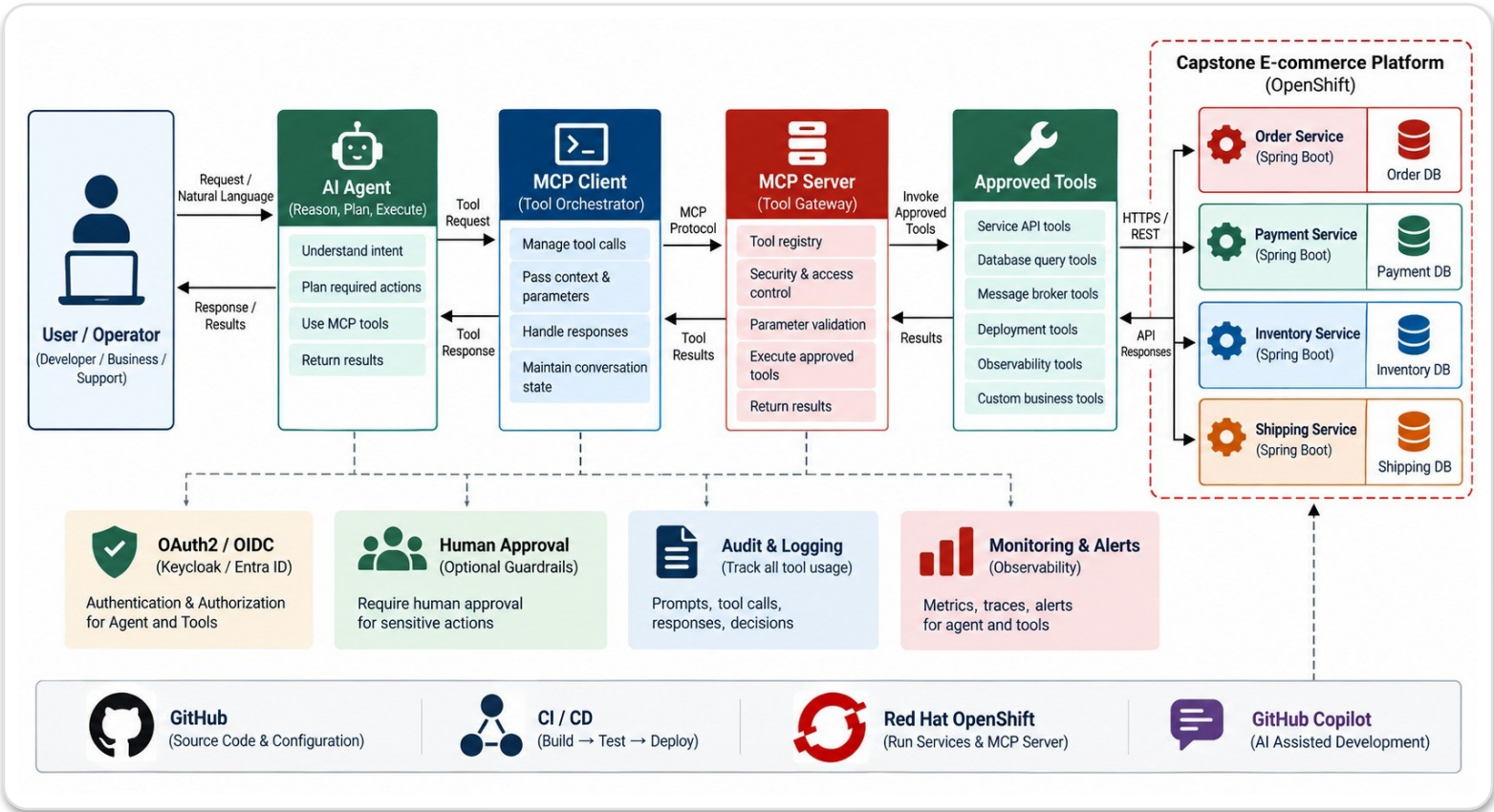
Review, test and scan like any other change.

BOUNDARY The developer owns the result.

KEY TAKEAWAY Use Copilot across the capstone — and review, test and secure everything it writes.

AGENTIC AI AND MCP INTEGRATION

Agent → MCP → backend microservice tools.



Example request

"Why is order ORD-1045 still pending?"

Tools

get_order, get_payment — read-only; refunds need approval.

NEVER

- ✗ Direct database access
- ✗ Superuser tokens

KEY TAKEAWAY Agents reach microservices only through MCP tools and APIs — never their databases.

THE CAPSTONE DEMO IN COMMANDS

Lab 5's script: prove the decisions locally, then prove the platform on OpenShift.

Risk decisions (PowerShell, labs/day-05/lab5)

```
$OPS = python tools\issue-jwt.py ops
$REVIEWER = python tools\issue-jwt.py reviewer

.\tools\publish-event.ps1 -File tools\events\approve.json
.\tools\publish-event.ps1 -File tools\events\approve.json # duplicate
.\tools\publish-event.ps1 -File tools\events\timeout.json # HOLD
.\tools\publish-event.ps1 -File tools\events\hold.json

curl.exe -s -H "Authorization: Bearer $OPS" `
  "http://localhost:8083/api/v1/assessments?disposition=HOLD"

# review TXN-B2C3D4E5 with $REVIEWER -> 200
# same POST with $OPS -> 403
```

Duplicate

Logs show it ignored — still one row.

Timeout

HOLD MODEL_UNAVAILABLE — never APPROVE.

Evidence

Route readiness 200, then a working rollback.

✓ OpenShift evidence and recovery

```
$PROJECT = oc project -q
oc -n $PROJECT rollout status deploy/risk-assessment-service

$RISK = oc -n $PROJECT get route risk-assessment-service `
  -o jsonpath="{.spec.host}"
curl.exe -s https://$RISK/actuator/health/readiness # 200

# Lab 4 recovery
oc -n $PROJECT rollout undo deploy/account-service
oc -n $PROJECT rollout status deploy/account-service
```

KEY TAKEAWAY Show the happy path, the failures and the recovery — each with evidence on screen.

CAPSTONE DEMONSTRATION — DAY 5

Official checkpoint: demonstrate Labs 1–5 as one working platform.

OFFICIAL CAPSTONE DEMO

30 MIN

labs/day-05/lab5/LAB-5-GUIDE.md

SCENARIO

Teams demonstrate the banking Account, Transaction and Risk Assessment services. Do not run TEKsystems Lab 13.

YOUR TASKS

- 1 Successful transaction → APPROVE
- 2 Duplicate event → still one row
- 3 Model timeout → HOLD, never APPROVE
- 4 Human review: reviewer 200, ops 403
- 5 OpenShift Route readiness 200
- 6 Rollback: oc rollout undo

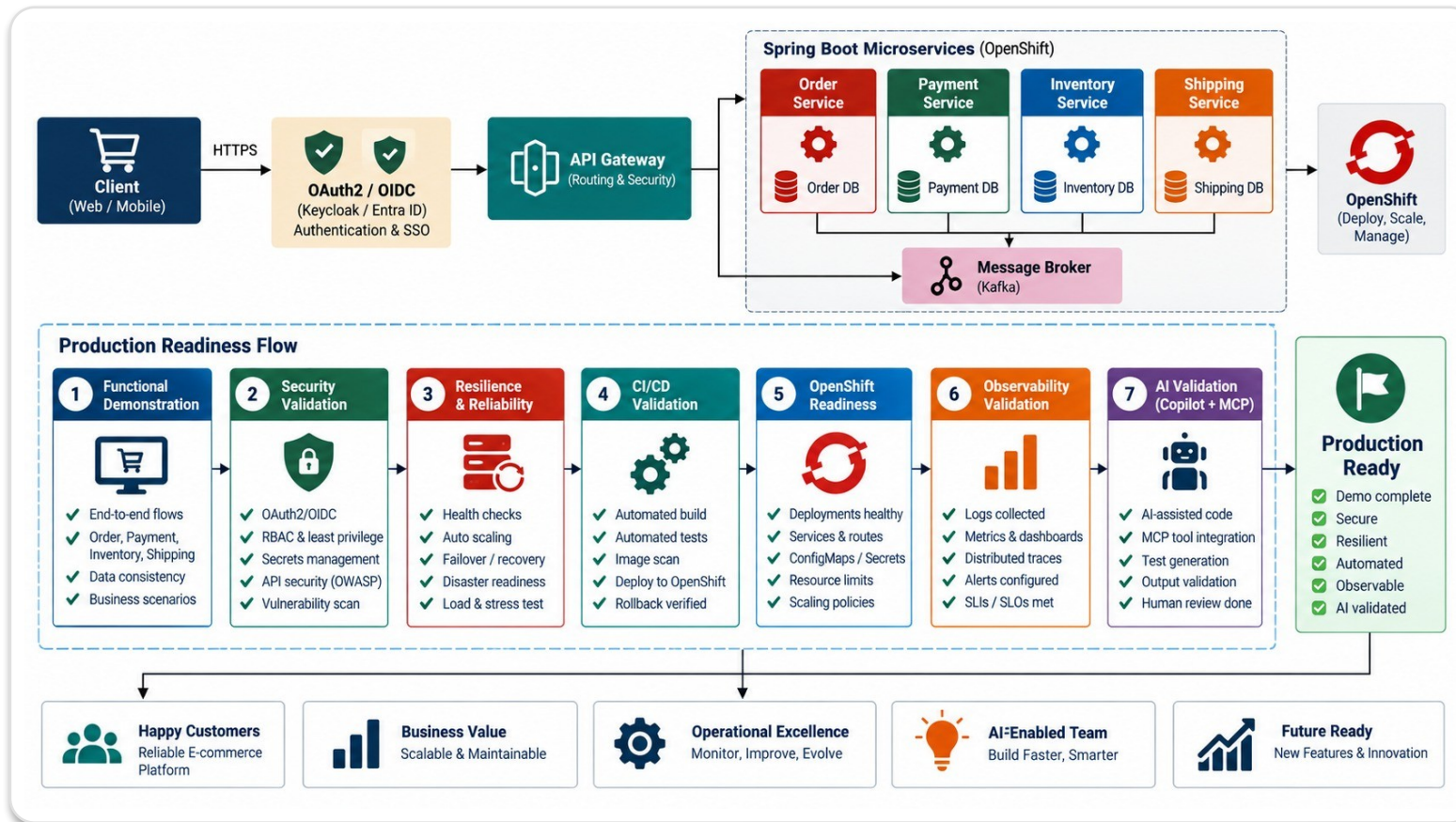
EXPECTED OUTCOME

- ✓ All six demos with visible evidence
- ✓ Synthetic, secret-free logs
- ✓ Each step explained by its module
- ✓ No invented financial outcome

KEY TAKEAWAY Six demos, each with evidence: success, duplicate, HOLD, review, OpenShift, recovery.

CAPSTONE DEMO AND PRODUCTION READINESS REVIEW

Demonstrate the story, then decide: is it ready?



READINESS QUESTIONS

- ✓ Correct under failure
- ✓ Secured and audited
- ✓ Delivered by pipeline
- ✓ Observable and recoverable
- ✓ AI kept under control

FINAL QUESTION Would you run this with real money?

KEY TAKEAWAY Demonstrate the whole engineering story — from order to operations — not one endpoint.

EXERCISE: PRODUCTION READINESS REVIEW

A team says their platform is ready. Review the evidence.

PRACTICE EXERCISE

10 MIN

SCENARIO

The team's readiness checklist:

- ✓ mvn verify passes
- ✓ Image pushed as :latest
- ✓ Route readiness returns 200
- ✓ JWT scopes on every endpoint
- DB password in the ConfigMap
- No alert on the HOLD backlog
- Rollback never tried

YOUR TASKS

- 1 Mark each item: ready, risky or blocking
- 2 Give the fix for each gap
- 3 Decide: go or no-go

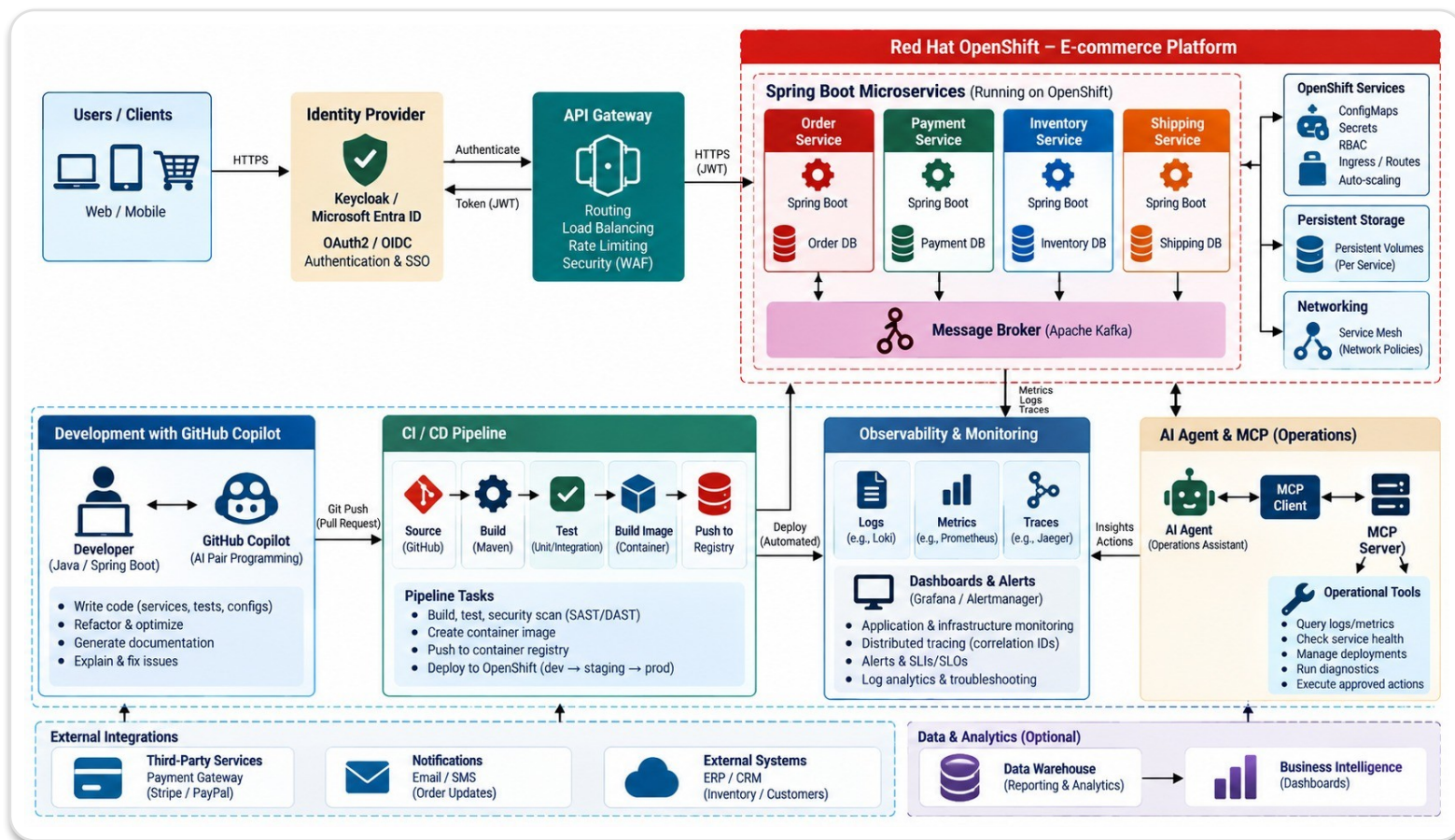
EXPECTED OUTCOME

- ✓ :latest → versioned tag and digest
- ✓ Password → Secret, then rotate
- ✓ Alert on HOLD backlog and DLQ
- ✓ Rehearse oc rollout undo
- ✓ No-go until the blockers are fixed

KEY TAKEAWAY Ready means secure, traceable, observable and recoverable — not just "it starts".

COMPLETE END-TO-END PRODUCTION ARCHITECTURE

Every module combined in one production platform.



Customer request path

Gateway, identity, services, events, data.

Software delivery path

Git, pipeline, registry, OpenShift.

Operations path

Probes, logs, metrics, traces, alerts.

Two AI patterns

Copilot for developers; agents through MCP.

KEY TAKEAWAY Production microservices = clear boundaries, security, automation and operations — with AI working through them.

CAPSTONE MISCONCEPTIONS AND ANTI-PATTERNS

Beliefs and habits that stop a working demo from being a production platform.

MISCONCEPTION

REALITY

ANTI-PATTERNS TO AVOID

It runs, so it's done

Deployed, observable and recoverable

A shared database is simpler

Share through APIs and events

Retries fix failures

Only with idempotency and limits

AI wrote the tests, so it's tested

Review every generated assertion

Agents can query the database

Only through MCP tools and APIs

- ✗ A distributed monolith
- ✗ :latest images in production
- ✗ Secrets in ConfigMaps
- ✗ Auto-approving on model failure
- ✗ Demoing only the happy path
- ✗ Rollback never rehearsed

REMEMBER Prove it with evidence — including failure and recovery.

KEY TAKEAWAY Production-ready means correct under failure, secure, automated and observable.

KNOWLEDGE CHECK (1 OF 2)

Design, data and resilience — answer in your own words.

1 Why should each microservice own its data?

2 When should services communicate through REST?

3 When should services communicate through events?

4 Why prefer local transactions over distributed ones?

5 What could Redis add to the platform?

6 How do timeouts and circuit breakers help?

7 What does CI validate before deployment?

KNOWLEDGE CHECK (2 OF 2)

Delivery, security, AI and operations.

1 Why are container images useful for OpenShift?

2 OpenShift Service vs. Route?

3 How does Spring Security protect APIs?

4 Why must secrets be externalized?

5 How can Copilot help without replacing review?

6 What does MCP provide for AI agents?

7 Which signals are needed for production readiness?

COURSE SUMMARY AND NEXT STEPS

From one Spring Boot service to a secure, observable, AI-assisted platform on OpenShift.

YOU CAN NOW

- ✓ Design service boundaries, APIs and events
- ✓ Build resilient, secure Spring Boot services
- ✓ Deliver them through CI/CD to OpenShift
- ✓ Observe, troubleshoot and recover them
- ✓ Use Copilot, agents and MCP under control

NEXT: BRING IT BACK TO YOUR TEAM

- 1 Which service would you harden first?
- 2 Which pipeline gate is missing today?
- 3 Where could an agent help safely?

TAKE WITH YOU Your labs repo, the readiness checklist and the demo script.

KEY TAKEAWAY Clear boundaries, built-in security, automation and observability — with AI working through them.